

INFORMATIQUE • 1ÈRE D

MODULE 2 : ALGORITHMIQUE ET PROGRAMMATION

LEÇON N09

PROGRAMMATION EN
JAVASCRIPT

DURÉE CONSEILLÉE : 7 H

DANS CETTE LEÇON

Cette leçon initie les élèves de 1re C, D, E à la programmation client-côté en JavaScript : variables, opérateurs, structures de contrôle, fonctions, événements et entrées/sorties, pour créer des pages web dynamiques et des convertisseurs pratiques.

- À la fin de cette leçon, je sais expliquer les limites du HTML statique et les apports de JavaScript pour l'interactivité.
- Je sais insérer du code JavaScript dans une page HTML à l'aide de la balise `<script>`.
- Je sais déclarer des variables (`var`, `let`, `const`), identifier leurs types et effectuer des conversions avec `parseInt` et `parseFloat`.
- Je sais utiliser les opérateurs arithmétiques, de comparaison, logiques et d'affectation composée.
- Je sais traduire un algorithme séquentiel, conditionnel ou itératif en code JavaScript.
- Je sais déclarer une fonction, la chiamer et l'associer à l'événement `onclick` d'un élément HTML.

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Pourquoi JavaScript ? – Du HTML statique au Web dynamique
2. Variables, types de données et conversions
3. Opérateurs et expressions
4. Structures de contrôle : séquentielles, conditionnelles, itératives
5. Fonctions et gestion d'événements (onclick)
6. Entrées/Sorties et réalisation de convertisseurs pratiques
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais expliquer les limites du HTML statique et les apports de JavaScript pour l'interactivité.
- Je sais insérer du code JavaScript dans une page HTML à l'aide de la balise `<script>`.
- Je sais déclarer des variables (`var`, `let`, `const`), identifier leurs types et effectuer des conversions avec `parseInt` et `parseFloat`.
- Je sais utiliser les opérateurs arithmétiques, de comparaison, logiques et d'affectation composée.
- Je sais traduire un algorithme séquentiel, conditionnel ou itératif en code JavaScript.
- Je sais déclarer une fonction, la chiamer et l'associer à l'événement `onclick` d'un élément HTML.
- Je sais réaliser des entrées/sorties avec `prompt`, `alert`, `confirm` et `document.write`.
- Je sais concevoir et programmer un convertisseur (devises, températures, dates) complet et fonctionnel.

PRÉREQUIS

- Notions de base du HTML (balises, attributs, structure d'une page).
- Algorithmique élémentaire : variables, affectation, structures conditionnelles et boucles (vu en 2de).

- Manipulation simple du navigateur (ouvrir la console, rafraîchir une page).
- Connaissance des systèmes de numération (entiers, réels) et des unités de mesure courantes.

1 POURQUOI JAVASCRIPT ? – DU HTML STATIQUE AU WEB DYNAMIQUE

Situation d'accroche. Il est 17 h dans un cybercafé du quartier Nlongkak à Yaoundé. Un client, commerçant qui vient de recevoir un devis en euros, veut convertir instantanément le montant de ses achats en francs CFA. Le gérant lui ouvre une petite page web : le client tape le montant, clique sur un bouton « Convertir », et le résultat s'affiche immédiatement, sans que la page se recharge, sans attendre une réponse d'un serveur lointain. Le client, étonné, demande : « Mais comment la page a-t-elle su calculer toute seule ? »

Voilà notre problématique : **comment rendre une page HTML vivante et interactive, capable de réagir aux actions de l'utilisateur et d'effectuer des traitements, pour répondre à ce besoin quotidien ?** La réponse tient en un mot : **JavaScript**. Mais pour comprendre pourquoi ce langage est indispensable, il faut d'abord mesurer les limites du HTML que tu as étudié dans les leçons précédentes.

1.1 Rappel : le HTML, un langage d'affichage statique

Le **HTML** (HyperText Markup Language) est un langage de **balisage** : il décrit la structure et le contenu d'une page web (titres, paragraphes, tableaux, images, liens, formulaires). Le navigateur lit le fichier HTML de haut en bas et l'affiche. Une fois la page affichée, elle est **figée** : c'est ce qu'on appelle une page **statique**.

EXEMPLE Une page HTML purement statique

Voici la page du cybercafé écrite uniquement en HTML. Elle affiche un taux de change, mais elle ne peut rien calculer :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Cybercafé Nlongkak – Taux du jour</title>
</head>
<body>
  <h1>Bienvenue au Cybercafé Nlongkak</h1>
  <p>Taux du jour : 1 euro = 655,957 FCFA</p>
  <p>Pour convertir, prenez votre calculatrice !</p>
</body>
</html>
```

Le client doit sortir sa calculatrice : la page ne fait qu'afficher. Aucun clic, aucune saisie ne déclenche le moindre traitement.

1.2 Les limites du HTML face au besoin d'interactivité

Reprenons le besoin du client : saisir un montant, cliquer, obtenir un résultat. Analysons pour-

quoi le HTML seul échoue :

- **Pas de calcul.** HTML ne connaît ni l'addition, ni la multiplication, ni aucune opération. On ne peut pas lui demander de calculer $150 \times 655,957$.
- **Pas de réaction aux actions de l'utilisateur.** Un clic sur un bouton, la saisie dans une zone de texte, le déplacement de la souris : HTML peut afficher un bouton ou un champ, mais ne peut pas réagir à leur utilisation.
- **Pas de décision ni de répétition.** Impossible d'écrire « si le montant est négatif, afficher une erreur » ou « répéter pour chaque article du panier » : HTML n'a ni structure conditionnelle, ni boucle.
- **Rechargement obligatoire.** Sans JavaScript, la seule façon d'obtenir un nouveau contenu est de renvoyer les données vers un serveur et de recharger toute la page : aller-retour réseau, attente, consommation de forfait data — un vrai problème quand la connexion est lente ou coûteuse.

À RETENIR L'essentiel sur les limites du HTML

HTML décrit et affiche ; il ne calcule pas, ne décide pas et ne réagit pas. Pour rendre une page interactive, il faut lui adjoindre un véritable langage de programmation exécuté par le navigateur : JavaScript.

1.3 JavaScript : le langage qui rend la page vivante

DÉFINITION Script

Un **script** est une portion de code, écrite dans un langage de script comme **JavaScript**, qui est interprétée et exécutée par le navigateur web (Chrome, Firefox, Edge...) au moment du chargement de la page ou en réponse à une action de l'utilisateur. Contrairement à un programme compilé classique, le script n'a pas besoin d'être transformé au préalable en fichier exécutable : le navigateur lit le texte du code et l'exécute directement.

JavaScript est donc un langage de **programmation** complet : variables, opérateurs, structures conditionnelles, boucles, fonctions — tout ce que tu as découvert en algorithmique s'y retrouve. Sa particularité : il vit à l'intérieur de la page web et peut la modifier en temps réel.

Le mécanisme clé : le DOM. Quand le navigateur charge une page HTML, il construit en mémoire une représentation de la page sous forme d'arborescence d'objets appelée **DOM** (Document Object Model). Chaque balise (titre, paragraphe, bouton, champ de saisie) devient un « objet » que JavaScript peut lire et modifier : changer un texte, récupérer une valeur saisie, afficher un résultat. C'est la **manipulation du DOM** qui permet à la page de réagir sans se recharger.

PROPRIÉTÉ Avantages et limites de JavaScript

Avantages :

- Exécution côté client : le code tourne dans le navigateur de l'utilisateur, ce qui évite les allers-retours vers le serveur. La réponse est immédiate, même avec une connexion lente.
- Interactivité immédiate : la page réagit aux clics, aux saisies, aux déplacements de souris grâce aux **événements**.
- Manipulation du DOM : le script peut modifier le contenu et l'apparence de la page sans rechargement.
- Simplicité de mise en œuvre : aucun logiciel à installer, un simple éditeur de texte et un navigateur suffisent.

Limites :

- Sécurité : pour protéger l'utilisateur, JavaScript exécuté dans une page ne peut pas accéder aux fichiers locaux du disque dur, ni lire d'autres documents de l'ordinateur.
- Dépendance au navigateur : le code peut se comporter légèrement différemment selon le navigateur, et l'utilisateur peut désactiver JavaScript.
- Code visible : n'importe qui peut afficher le code source de la page et lire le script ; on n'y place donc jamais de secret (mot de passe, clé...).

ATTENTION JavaScript n'est pas Java !

Malgré la ressemblance des noms, JavaScript et Java sont deux langages totalement différents, ni compatibles ni interchangeables. JavaScript s'exécute dans le navigateur ; Java est un langage compilé à usage général. Ne confonds jamais les deux dans une copie d'examen !

LE SAVAIS-TU ? Le moteur V8 : JavaScript à la vitesse de l'éclair

On dit que JavaScript est « interprété », mais les navigateurs modernes font bien mieux. Le moteur V8 de Google Chrome (aussi utilisé par Edge et Node.js) analyse le code JavaScript et le compile en code machine à la volée pendant l'exécution : c'est la compilation JIT (Just-In-Time). Résultat : des scripts exécutés presque aussi vite qu'un programme natif. Né en 1995 en seulement dix jours sous le nom de LiveScript, JavaScript est aujourd'hui le langage le plus utilisé au monde — y compris au Cameroun, où il anime les plateformes de services en ligne et les applications mobiles hybrides.

1.4 Insérer un script dans une page HTML : la balise `<script>`

Pour intégrer du JavaScript dans une page, on utilise la balise `<script>...</script>`. Deux questions se posent alors : où placer cette balise, et quand le script sera-t-il exécuté ?

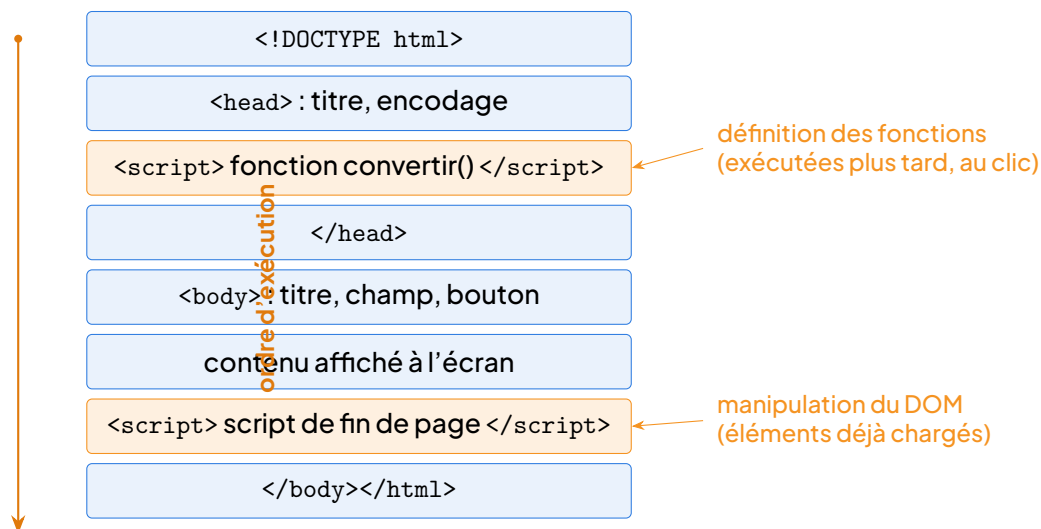
Deux emplacements possibles :

1. **Dans l'en-tête `<head>`** : le script est chargé avant le contenu de la page. C'est l'endroit naturel pour déclarer des fonctions qui seront appelées plus tard (par exemple au clic sur un bouton).
2. **En fin de `<body>`**, juste avant `</body>` : le script n'est exécuté qu'après l'affichage de tout le contenu. C'est l'endroit idéal pour un script qui manipule directement les éléments de la page, car ils existent déjà dans le DOM.

EXEMPLE Premier script : notre convertisseur réagit enfin

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Convertisseur FCFA - Euro</title>
  <script>
    // Fonction appelée au clic sur le bouton
    function convertir() {
      var montant = parseFloat(document.getElementById("euro").value);
      var resultat = montant * 655.957;
      document.getElementById("sortie").innerHTML =
        montant + " euro(s) = " + resultat + " FCFA";
    }
  </script>
</head>
<body>
  <h1>Convertisseur du Cybercafé Nlongkak</h1>
  <label>Montant en euros :</label>
  <input type="text" id="euro">
  <button onclick="convertir()">Convertir</button>
  <p id="sortie"></p>
</body>
</html>
```

Le script est placé dans le `<head>` : la fonction `convertir()` y est définie mais ne s'exécute que lorsque l'utilisateur clique sur le bouton, grâce à l'événement `onclick`. Nous détaillerons tout ce mécanisme (variables, conversion de type, fonctions, événements, manipulation du DOM) dans les sections 2, 5 et 6.



Le navigateur lit la page de haut en bas : le script du `<head>` est chargé avant l'affichage du contenu ; le script placé en fin de `<body>` s'exécute une fois tous les éléments de la page construits dans le DOM.

1.5 Le problème du blocage : les attributs `defer` et `async`

Un script placé dans le `<head>` sans précaution possède un défaut : le navigateur, en le rencontrant, **interrompt l'affichage de la page** pour télécharger et exécuter le script, puis reprend. On dit que le script est **bloquant**. Si le script est volumineux ou le réseau lent, l'utilisateur patiente devant une page blanche.

Deux attributs de la balise `<script>` résolvent ce problème :

- **defer** : le script est téléchargé en parallèle du chargement de la page, mais n'est exécuté qu'**après** la construction complète du DOM, dans l'ordre d'apparition des scripts.
- **async** : le script est téléchargé en parallèle et exécuté **dès qu'il est prêt**, sans attendre le reste de la page (à réserver aux scripts indépendants, comme les statistiques de visite).

MÉTHODE Bien placer son script

Pour garantir que le script trouve tous les éléments de la page sans bloquer l'affichage :

1. Si le script est dans un fichier externe et manipule la page : place la balise dans le `<head>` avec l'attribut `defer` : `<script src="app.js" defer></script>`.
2. Sinon (petit script intégré) : place la balise `<script>` juste avant `</body>`.
3. Réserve `async` aux scripts totalement indépendants du contenu de la page.
4. Évite les scripts lourds sans `defer` ni `async` dans le `<head>`.

ATTENTION Erreur fréquente : le script qui bloque la page

Oublier `defer` ou `async` sur un script volumineux placé dans le `<head>` bloque l’affichage : la page reste blanche pendant le téléchargement. Autre piège lié : un script exécuté trop tôt qui cherche un élément pas encore chargé (par exemple `document.getElementById("euro")` avant que le champ existe) provoque une erreur et le reste du script ne s’exécute pas. D’où la règle d’or : `defer` dans le `<head>`, ou script en fin de `<body>`.

EXEMPLE Exemple chiffré : l’impact sur le temps de chargement

Le gérant du cybercafé a mesuré le temps d’affichage complet de sa page sur la connexion 3G du quartier :

Configuration de la page	Temps d’affichage
Page sans JavaScript	0,8 s
Script bloquant (300 ko) dans le <code><head></code> sans attribut	1,5 s
Même script avec <code>defer</code>	0,9 s

Le script bloquant a presque doublé le temps d’attente ($1,5 - 0,8 = 0,7$ s de blocage), alors qu’avec `defer` le surcoût n’est plus que de 0,1 s : le téléchargement se fait en parallèle de l’affichage. Sur un forfait data limité et une connexion instable, ce gain est décisif pour les clients du cybercafé.

1.6 Bonne pratique : séparer le JavaScript dans un fichier .js

Comme en CSS, il est vivement recommandé de **séparer** le code JavaScript du code HTML : on écrit le script dans un fichier externe d’extension `.js`, puis on l’inclut dans la page avec l’attribut `src`.

EXEMPLE Code séparé : fichier externe

Fichier `convertisseur.js` :

```
// Fichier convertisseur.js : logique de l'application
function convertir() {
    var montant = parseFloat(document.getElementById("euro").value);
    var resultat = montant * 655.957;
    document.getElementById("sortie").innerHTML =
        montant + " euro(s) = " + resultat + " FCFA ";
}
```

Fichier `index.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Convertisseur FCFA – Euro</title>
  <script src="convertisseur.js" defer></script>
</head>
<body>
  <h1>Convertisseur du Cybercafé Nlongkak</h1>
  <input type="text" id="euro">
  <button onclick="convertir()">Convertir</button>
  <p id="sortie"></p>
</body>
</html>
```

Avantages : le code est plus lisible, réutilisable dans plusieurs pages, et le navigateur peut le mettre en cache (chargement plus rapide lors des visites suivantes). Remarque : une balise `<script>` avec un attribut `src` doit avoir un corps vide.

EXERCICE RÉSOLU 1**Choisir le bon emplacement de script**

Énoncé. Le gérant du cybercafé écrit une page contenant un grand script de 250 ko qui, dès le chargement, colore en vert tous les prix affichés dans un tableau. Il place la balise `<script src="prix.js"></script>` dans le `<head>`, sans attribut. Ses clients se plaignent que la page met longtemps à s'afficher, et parfois les prix ne sont pas colorés. Expliquer les deux problèmes et proposer une correction.

SOLUTION

Solution détaillée.

1. Problème 1 — blocage de l'affichage. Sans `defer` ni `async`, le navigateur suspend l'affichage de la page le temps de télécharger et d'exécuter les 250 ko du script : d'où la page blanche et l'attente constatées par les clients.
2. Problème 2 — DOM pas encore construit. Le script s'exécute avant que le tableau des prix n'existe dans le DOM. S'il cherche les cellules du tableau trop tôt, il ne les trouve pas et la coloration échoue.
3. Correction. Deux solutions équivalentes :
 - ajouter l'attribut `defer` : `<script src="prix.js" defer></script>` dans le `<head>` — le téléchargement se fait en parallèle et l'exécution attend la fin de la construction du DOM;
 - ou déplacer la balise juste avant `</body>` : `<script src="prix.js"></script>`.

Dans les deux cas, la page s'affiche sans blocage et le tableau existe lorsque le script

s'exécute : les prix sont correctement colorés en vert.

EXPÉRIENCE TP guidé : observer l'ordre d'exécution des scripts

Objectif. Vérifier concrètement qu'un script du `<head>` s'exécute avant le contenu du `<body>`, et qu'un script en fin de `<body>` trouve bien les éléments de la page.

Matériel. Un ordinateur avec un éditeur de texte (Notepad++, Sublime Text ou Bloc-notes) et un navigateur (Chrome ou Firefox).

Manipulation.

1. Crée un fichier `test.html` contenant le code ci-dessous :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Test d'exécution</title>
  <script>
    alert("1. Script du head : la page n'est pas encore affichée");
  </script>
</head>
<body>
  <h1 id="titre">Bonjour la Première !</h1>
  <script>
    alert("2. Script de fin de body : " +
      document.getElementById("titre").innerHTML);
  </script>
</body>
</html>
```

2. Enregistre le fichier, puis ouvre-le avec le navigateur (double-clic).
3. Observe l'ordre d'apparition des deux boîtes de dialogue et l'état de la page derrière chacune.
4. Déplace maintenant le premier `<script>` (avec son `alert`) juste avant `</body>` et recharge : observe la différence d'affichage.

Observations attendues. La première boîte de dialogue apparaît alors que la page est encore blanche (le titre n'est pas affiché) : le script du `<head>` s'exécute avant l'affichage du corps. La deuxième boîte affiche « Bonjour la Première ! » : le script de fin de `<body>` accède sans erreur à l'élément `titre`, car celui-ci existe déjà dans le DOM.

Interprétation. Le navigateur exécute les scripts dans l'ordre de lecture du fichier, de haut en bas. Un script qui manipule des éléments de la page doit donc être placé après eux (fin de `<body>`) ou chargé avec `defer`.

EXERCICE 1 Pour t'entraîner

1. Cite trois limites du HTML qui justifient l'utilisation de JavaScript.
2. Vrai ou faux (justifie) : « Un script JavaScript exécuté dans une page web peut lire le contenu du disque dur de l'utilisateur. »
3. Quelle est la différence entre les attributs `defer` et `async` de la balise `<script>` ?
4. Le gérant veut réutiliser son script de conversion sur trois pages différentes de son site. Quelle bonne pratique doit-il adopter et pourquoi ?

CORRIGÉ Exercice

1. HTML ne sait pas calculer, ne peut pas réagir aux actions de l'utilisateur (clics, saisies) sans recharger la page, et ne possède ni structure conditionnelle ni boucle.
2. Faux. Pour des raisons de sécurité, JavaScript s'exécute dans un environnement restreint (le navigateur) et n'a pas accès aux fichiers locaux du système.
3. `defer` : le script est téléchargé en parallèle et exécuté après la construction complète du DOM, dans l'ordre des balises. `async` : le script est exécuté dès qu'il est téléchargé, sans attendre le reste de la page.
4. Il doit placer le code dans un fichier externe `.js` (par exemple `convertisseur.js`) et l'inclure dans chaque page avec `<script src="convertisseur.js" defer></script>`. Ainsi, une seule copie du code est maintenue, réutilisée partout et mise en cache par le navigateur.

À RETENIR Bilan de la section

- HTML seul produit des pages statiques : pas de calcul, pas de réaction, pas de logique.
- JavaScript est un langage de programmation exécuté côté client, dans le navigateur ; il rend la page interactive et manipule le DOM.
- Un script s'insère avec la balise `<script>`, dans le `<head>` (avec `defer`) ou en fin de `<body>`.
- Sans `defer/async`, un script du `<head>` bloque l'affichage ; avec `defer`, la page reste rapide et le DOM est prêt.
- Bonne pratique : séparer le code dans un fichier `.js` inclus avec l'attribut `src`.

Dans la prochaine section, nous apprendrons à déclarer des variables en JavaScript et à

convertir les types de données — l'étape indispensable pour construire notre convertisseur complet.

2 VARIABLES, TYPES DE DONNÉES ET CONVERSIONS

Dans la section précédente, nous avons vu que le HTML seul ne sait produire que des pages statiques : une fois affichée, la page ne réagit plus à l'utilisateur. JavaScript vient combler cette lacune en permettant à la page de calculer, de mémoriser et de réagir. Mais pour calculer, il faut d'abord pouvoir **stocker des valeurs** : le total d'une facture dans une boutique de Douala, la note d'un élève, le montant en FCFA à convertir en euros. C'est précisément le rôle des **variables**. Cette section est donc le véritable point de départ de la programmation JavaScript : sans variables, aucun script ne peut exister.

2.1 La notion de variable

DÉFINITION Variable

Une **variable** est un conteneur nommé qui stocke en mémoire une valeur (nombre, texte, vrai/faux, etc.) que le programme peut lire et réutiliser. Cette valeur est modifiable au cours de l'exécution si la variable est déclarée avec `let` ou `var` ; elle est non réaffectable (constante) si elle est déclarée avec `const`.

Imagine une boîte étiquetée dans la réserve d'une boutique : l'étiquette est le nom de la variable, le contenu de la boîte est sa valeur. On peut ouvrir la boîte pour lire le contenu, ou remplacer le contenu par un autre. En algorithmique, tu as déjà rencontré cette idée avec l'instruction `Variable x : Entier`. JavaScript reprend le même concept, avec sa propre syntaxe.

2.2 Déclaration des variables : `var`, `let` et `const`

PROPRIÉTÉ Syntaxe de déclaration

En JavaScript, on déclare une variable avec l'un des trois mots-clés suivants, suivis du nom de la variable et éventuellement d'une valeur initiale :

- `var nom = valeur;` : portée fonction (la variable est visible dans toute la fonction où elle est déclarée). C'est la forme historique, aujourd'hui déconseillée car source de confusions.
- `let nom = valeur;` : portée bloc (la variable n'existe qu'entre les accolades `{ }` où elle est déclarée), valeur réaffectable. C'est la forme moderne recommandée pour les variables ordinaires.

- `const nom = valeur;` : portée bloc, valeur non réaffectable : l'initialisation est obligatoire et toute nouvelle affectation provoque une erreur.

Chaque instruction se termine par un point-virgule ; (fortement recommandé).

EXEMPLE Déclarer et utiliser des variables

```
let nomEleve = "Amina"; // variable texte, modifiable
let age = 16;           // variable numerique, modifiable
const TAUX_TVA = 19.25; // constante : ne changera jamais

age = age + 1;           // reaffectation autorisee avec let
// TAUX_TVA = 20;       // ERREUR : impossible de reaffecter une const
var ancien = "style var"; // ancienne syntaxe, a eviter
```

ATTENTION Règles de nommage des variables

Le nom d'une variable JavaScript doit respecter des règles strictes :

- il commence obligatoirement par une lettre, un souligné `_` ou un dollar `$` ; jamais par un chiffre (`2notes` est interdit) ;
- il ne contient ni espace, ni tiret, ni accent (préfère `noteMaths` à `note maths`) ;
- JavaScript est sensible à la casse : `moyenne`, `Moyenne` et `MOYENNE` sont trois variables différentes ;
- les mots réservés du langage sont interdits : `let`, `const`, `var`, `function`, `if`, `else`, `for`, `while`, `return`, etc.

Par convention, on utilise la notation camelCase : première lettre minuscule, puis une majuscule à chaque nouveau mot : `montantTotal`, `nombreEleves`.

2.3 Les types primitifs de données

Contrairement à l'algorithmique scolaire où l'on déclare `Entier`, `Réel` ou `Chaîne`, JavaScript détermine **automatiquement** le type d'une variable à partir de la valeur qu'on lui affecte. On distingue sept **types primitifs**.

DÉFINITION Types primitifs

Un **type primitif** est une catégorie de valeurs simples (non composées d'objets). JavaScript en définit sept :

- **Number** : tous les nombres, entiers ou décimaux (42, 3.14, -7);
- **String** : les chaînes de caractères, entre guillemets simples ou doubles ("Bonjour");
- **Boolean** : les valeurs logiques `true` ou `false`;
- **Undefined** : valeur d'une variable déclarée mais jamais initialisée;
- **Null** : absence volontaire de valeur (`let x = null;`);
- **Symbol** : identifiant unique (usage avancé);
- **BigInt** : entiers de taille arbitraire, notés avec un `n` final (9007199254740993n).

L'opérateur `typeof` permet de **vérifier le type** d'une valeur ou d'une variable : il renvoie une chaîne indiquant le type.

Type	Exemple de littéral	Résultat de <code>typeof</code>
Number	42, 3.14, -7	"number"
String	"Bonjour", 'FCFA'	"string"
Boolean	true, false	"boolean"
Undefined	let x; (non initialisée)	"undefined"
Null	null	"object" (anomalie historique)
Symbol	Symbol("id")	"symbol"
BigInt	12345678901234567890n	"bigint"

Tableau récapitulatif des sept types primitifs de JavaScript, avec un exemple de littéral et le résultat de l'opérateur `typeof`.

ATTENTION Le piège de `typeof null`

Par une erreur historique conservée pour compatibilité, `typeof null` renvoie `"object"` et non `"null"`. Pour tester si une variable vaut `null`, on écrit directement `x === null`.

2.4 Le typage dynamique

DÉFINITION Typage dynamique

JavaScript est un langage à **typage dynamique** : le type d'une variable n'est pas fixé à la

déclaration ; il dépend de la valeur qu'elle contient et peut changer au cours de l'exécution.

EXEMPLE Une variable qui change de type

```
let donnee = 25;           // donnee est un Number
alert(typeof donnee);      // affiche "number"

donnee = "vingt-cinq";     // la meme variable devient une String
alert(typeof donnee);      // affiche "string"

donnee = true;             // puis un Boolean
alert(typeof donnee);      // affiche "boolean"
```

Ce comportement est souple, mais dangereux : si tu perds de vue le type réel d'une variable, les opérations peuvent produire des résultats surprenants (voir la conversion implicite ci-dessous). En algorithmique classique, au contraire, le type est fixé une fois pour toutes à la déclaration.

2.5 Les conversions de type

Pourquoi convertir ? Parce que **toute saisie utilisateur** (via `prompt()` ou une zone de texte) arrive sous forme de **chaîne de caractères**. Si l'utilisateur tape 5000 pour un montant en FCFA, JavaScript reçoit la chaîne "5000", pas le nombre 5000. Avant tout calcul, il faut donc convertir.

Conversion explicite

PROPRIÉTÉ Fonctions de conversion explicite

- `parseInt(chaine, base)` : convertit une chaîne en nombre entier. Le second argument est la base de numération (10 pour le décimal — toujours le préciser). La lecture s'arrête au premier caractère non numérique.
- `parseFloat(chaine)` : convertit une chaîne en nombre décimal.
- `Number(valeur)` : conversion générale en nombre ; renvoie `NaN` si la chaîne n'est pas entièrement numérique.
- `String(valeur)` : convertit en chaîne de caractères.
- `Boolean(valeur)` : convertit en booléen ; les valeurs 0, "" (chaîne vide), `null`, `undefined` et `NaN` deviennent `false`, tout le reste devient `true`.

MÉTHODE Convertir une saisie utilisateur

Pour récupérer un nombre saisi par l'utilisateur et pouvoir calculer avec :

1. Appeler `prompt()` qui renvoie toujours une chaîne (ou `null` si l'utilisateur annule).
2. Convertir immédiatement avec `parseFloat()` (nombre décimal) ou `parseInt(chaine, 10)` (entier).
3. Tester le résultat avec `isNaN()` pour détecter une saisie invalide.

Modèle à retenir : `let n = parseFloat(prompt('Entrez un nombre:'));`

ATTENTION Erreurs fréquentes avec `parseInt`

`parseInt('10px', 10)` renvoie 10 : la fonction lit les chiffres tant qu'elle le peut, puis s'arrête. En revanche, `parseInt('abc', 10)` renvoie `NaN` (Not a Number) car aucun chiffre ne se trouve au début. Toujours tester le résultat avec `isNaN()` avant de calculer :

```
let n = parseInt(prompt('Entrez un entier:'), 10);
if (isNaN(n)) {
  alert('Saisie invalide !');
} else {
  alert('Le double est ' + (2 * n));
}
```

Autre piège : `parseInt('3.99', 10)` renvoie 3 (troncature, pas arrondi). Pour un montant en FCFA avec décimaux, utilise `parseFloat()`.

Conversion implicite

JavaScript convertit parfois les types **automatiquement**, sans qu'on le demande. C'est la **conversion implicite** (ou coercion), source des bugs les plus classiques du débutant.

PROPRIÉTÉ Priorité de conversion

- Avec l'opérateur `+`, si l'un des deux opérandes est une chaîne, JavaScript convertit l'autre en chaîne et effectue une concaténation (collage des textes).
- Avec les opérateurs `-`, `*` et `/`, JavaScript convertit toujours les chaînes numériques en `Number` et effectue le calcul arithmétique.

EXEMPLE Exemple chiffré : le piège du `+`

```
let a = '5'; // a est une CHAÎNE
let b = 2;   // b est un NOMBRE

alert(a + b); // affiche "52" : concatenation, pas addition !
alert(a - b); // affiche 3 : '5' est converti en 5, puis 5 - 2
alert(a * b); // affiche 10
alert(a / b); // affiche 2.5
alert(b + b); // affiche 4 : deux nombres, vraie addition
```

Explication pas à pas de `a + b` : JavaScript voit que `a` est une chaîne ; il convertit donc `b` en chaîne "2" et colle les deux textes : "5" + "2" donne "52". Pour obtenir 7, il fallait écrire `parseFloat(a) + b`.

À RETENIR L'essentiel des conversions

- Une saisie `prompt()` est toujours une chaîne : convertis-la avant tout calcul.
- `+` avec une chaîne $\Rightarrow$ concaténation ; `-`, `*`, `/` $\Rightarrow$ conversion en nombre.
- `typeof` permet de vérifier le type d'une variable à tout moment.
- `isNaN()` permet de valider une conversion numérique.

LESAVAIS-TU ? `const` ne rend pas tout immuable

Beaucoup d'élèves croient qu'une variable déclarée `const` est totalement figée. En réalité, `const` verrouille seulement la liaison entre le nom et la valeur : on ne peut pas réaffecter le nom. Mais si la valeur est un objet ou un tableau, son contenu peut changer !

```
const eleve = { nom : "Amina", note : 14 };
eleve.note = 17; // AUTORISE : on modifie une propriété
// eleve = { nom : "Bob" }; // ERREUR : réaffectation interdite
```

C'est comme une salle de classe (`const salle`) : on ne peut pas changer de salle, mais on peut déplacer les tables à l'intérieur.

EXERCICE RÉSOLU 2 Calcul du montant TTC d'un article

Énoncé. Une boutique de Yaoundé vend un article hors taxe. Écris un script qui demande le prix hors taxe (en FCFA), applique une TVA constante de 19,25 %, et affiche le montant TTC. On veillera à convertir la saisie et à vérifier sa validité.

SOLUTION

Solution détaillée.

```
// Constante : le taux ne change jamais pendant l'exécution
const TAUX_TVA = 19.25;

// 1. Saisie : prompt() renvoie une chaîne, on convertit en nombre
let prixHT = parseFloat(prompt('Prix hors taxe (FCFA) :'));

// 2. Vérification de la validité de la saisie
if (isNaN(prixHT)) {
  alert('Erreur : veuillez entrer un nombre valide. ');
} else {
  // 3. Calcul : prixHT est bien un Number, pas de concaténation
  let montantTTC = prixHT * (1 + TAUX_TVA / 100);
  // 4. Affichage : ici le + est une concaténation volontaire
  alert('Montant TTC : ' + montantTTC + ' FCFA');
}
```

Trace d'exécution pour une saisie de 10000 : prompt renvoie la chaîne "10000", parseFloat la convertit en 10000 ; le calcul donne $10000 \times 1,1925 = 11925$; l'alerte affiche « Montant TTC : 11925 FCFA ». Sans parseFloat, l'expression `prixHT * (1 + TAUX_TVA/100)` fonctionnerait encore (conversion implicite de *), mais `prixHT + 1925` aurait produit la chaîne "100001925" : d'où l'importance de convertir systématiquement dès la saisie.

EXERCICE 2 À toi de jouer

1. Déclare une constante `TAUX_CHANGE` valant 655,957 (taux euro/FCFA) et une variable `montantEuro`. Demande à l'utilisateur un montant en euros, convertis-le correctement, puis affiche l'équivalent en FCFA.
2. Prédise le résultat de chaque expression, puis vérifie dans la console du navigateur : `'3' + 4`, `'3' - 4`, `3 + '4' + 5`, `typeof NaN`, `Boolean('')`, `parseInt('12.9', 10)`.
3. Explique pourquoi `let 1erPrix = 500` ; provoque une erreur, et propose deux noms corrects.

CORRIGÉ Exercice : conversions et nommage

1. Script de conversion :

```
const TAUX_CHANGE = 655.957;
let montantEuro = parseFloat(prompt('Montant en euros :'));
```

```
if (isNaN(montantEuro)) {  
    alert('Saisie invalide.');
```

```
} else {
```

```
    let montantFCFA = montantEuro * TAUX_CHANGE;
```

```
    alert(montantEuro + ' EUR = ' + montantFCFA + ' FCFA');
```

```
}
```

2. '3' + 4 ⇒ "34" (concaténation); '3' - 4 ⇒ -1 (conversion en nombre); 3 + '4' + 5 ⇒ "345" (le premier + produit la chaîne "34", puis concaténation avec 5); typeof NaN ⇒ "number" (NaN est de type Number!); Boolean('') ⇒ false; parseInt('12.9', 10) ⇒ 12 (troncature).
3. Un nom de variable ne peut pas commencer par un chiffre. Noms corrects : premierPrix OU prix1.

Tu maîtrises désormais le stockage et la conversion des données : la matière première de tout script. Dans la section suivante, nous apprendrons à combiner ces valeurs grâce aux **opérateurs** arithmétiques, de comparaison et logiques, pour construire de véritables expressions de calcul.

3 OPÉRATEURS ET EXPRESSIONS

Dans la section précédente, nous avons appris à déclarer des variables et à y stocker des valeurs : un âge, un prix en FCFA, une température, un nom. Mais une variable qui ne sert qu'à stocker est comme une calculatrice dont on n'utiliserait que la mémoire : l'intérêt reste limité. Le véritable pouvoir d'un programme apparaît lorsqu'on combine ces valeurs : calculer le total d'une facture au cybercafé, vérifier qu'un élève est admissible (moyenne ≥ 10 et assiduité correcte), convertir des devises, incrémenter un compteur de visiteurs d'un site. Toutes ces actions reposent sur un même outil : les **opérateurs**, qui permettent de construire des **expressions**.

DÉFINITION Expression et opérateur

Une **expression** est une combinaison de valeurs, de variables et d'**opérateurs** que JavaScript évalue pour produire un résultat. Un **opérateur** est un symbole (ou un mot-clé) qui réalise une opération sur un ou plusieurs opérandes. Par exemple, dans 3 + 4, l'opérateur est + et les opérandes sont 3 et 4 ; l'expression s'évalue en 7.

3.1 Les opérateurs arithmétiques

Les opérateurs arithmétiques réalisent les calculs mathématiques usuels. Ce sont les premiers outils du programmeur : total d'une commande, moyenne des notes, conversion d'unités.

PROPRIÉTÉ Opérateurs arithmétiques de JavaScript

Opérateur	Nom	Exemple (résultat)
+	Addition	7 + 2 donne 9
-	Soustraction	7 - 2 donne 5
*	Multiplication	7 * 2 donne 14
/	Division	7 / 2 donne 3.5
%	Reste de la division (modulo)	7 % 2 donne 1
Math.pow(a,b)	Puissance (exponentiation)	Math.pow(2,3) donne 8

Deux de ces opérateurs méritent une attention particulière.

Le modulo %. Il renvoie le reste de la division entière. C'est l'outil roi pour tester la divisibilité : $n \% 2 === 0$ est vrai si et seulement si n est pair. Au marché de Mokolo, si tu veux répartir 53 cahiers dans des cartons de 10, $53 \% 10$ te donne les 3 cahiers restants.

La puissance Math.pow. Introduite récemment dans le langage, $\text{Math.pow}(a,b)$ calcule a^b : $\text{Math.pow}(5,2)$ vaut 25. Attention : la puissance est associative à droite, c'est-à-dire que $2^{(3^2)} = 2^9 = 512$, et non $(2^3)^2 = 64$.

EXEMPLE Calculs arithmétiques commentés

```
let prixUnitaire = 1500; // prix d'un cahier en FCFA
let quantite = 7;
let total = prixUnitaire * quantite; // 10500
let remise = total * 0.10; // 10 % de remise : 1050
let net = total - remise; // 9450
let reste = quantite % 2; // 1 : quantite impaire
let surface = 3 ** 2; // 9 (carre de cote 3)
console.log(net); // affiche 9450
```

ATTENTION Le piège du + avec les chaînes

L'opérateur + a un double rôle : il additionne les nombres mais concatène (colle) les chaînes de caractères. Ainsi "5" + 3 donne la chaîne "53", et non 8 ! C'est pourquoi les valeurs lues avec `prompt()` — toujours des chaînes — doivent être converties avec `parseInt()` ou `parseFloat()` avant tout calcul, comme nous l'avons vu à la section 2.

3.2 Affectation composée, incrémentation et décrémentation

Très souvent, on veut modifier une variable à partir de sa valeur actuelle : ajouter un montant à un solde, retirer un article du stock, compter les visiteurs. JavaScript offre des raccourcis pour cela.

PROPRIÉTÉ Affectations composées

Pour tout opérateur arithmétique $\oplus$, l'instruction $x \oplus= a$ est équivalente à $x = x \oplus a$. On dispose ainsi de $+=$, $-=$, $*=$, $/=$, $\%=$ et $**=$.

EXEMPLE Solde d'un compte mobile money

```
let solde = 5000; // solde initial en FCFA
solde += 2000;    // depot : solde = 7000
solde -= 1500;    // retrait : solde = 5500
solde *= 2;       // doublement (jeu) : solde = 11000
solde /= 11;      // partage : solde = 1000
solde %= 300;     // reste : solde = 100
```

Incrémentation et décrémentation. Le cas particulier « ajouter 1 » ou « retirer 1 » est si fréquent (compteurs, boucles) qu'il possède ses propres opérateurs : `++` et `--`. Ils existent en deux positions :

- **Postfixée** ($x++$) : la variable est incrémentée, mais l'expression vaut l'ancienne valeur ;
- **Préfixée** ($++x$) : la variable est incrémentée, et l'expression vaut la nouvelle valeur.

EXEMPLE Préfixe contre postfixe

```
let a = 5;
let b = a++; // b reçoit 5, PUIS a devient 6
let c = 5;
let d = ++c; // c devient 6, PUIS d reçoit 6
// Resultat : a = 6, b = 5, c = 6, d = 6
let e = 10;
e--;        // e = 9 (equivalent a e = e - 1)
```

ATTENTION Piège classique

Dans une instruction seule, `x++`; et `++x`; ont le même effet. La différence n'apparaît que lorsque l'expression est utilisée dans un calcul ou une affectation. Pour éviter toute confusion au Probatoire, utilise `x++` seul sur sa ligne et effectue les affectations séparément.

3.3 Les opérateurs de comparaison

Un programme interactif doit pouvoir comparer : la moyenne est-elle au moins égale à 10 ? Le mot de passe saisi est-il le bon ? Les comparaisons produisent un booléen : `true` (vrai) ou `false` (faux).

PROPRIÉTÉ Opérateurs de comparaison

Opérateur	Signification
<code>==</code>	Égalité simple (avec conversion de type)
<code>===</code>	Égalité stricte (même type et même valeur)
<code>!=</code>	Différent (avec conversion de type)
<code>!==</code>	Strictement différent
<code>></code> , <code><</code>	Supérieur, inférieur
<code>>=</code> , <code><=</code>	Supérieur ou égal, inférieur ou égal

ATTENTION `==` contre `===` : le piège de la coercition

L'opérateur `==` effectue une conversion de type (coercition) avant de comparer : `5 == "5"` vaut `true`, car la chaîne `"5"` est convertie en nombre ! Pire, `0 == false` et `"" == false` valent aussi `true`. L'opérateur `===`, lui, compare le type et la valeur : `5 === "5"` vaut `false`. Règle d'or : privilégie toujours `===` et `!==`. Et n'oublie jamais qu'un seul `=` est une affectation, pas une comparaison : écrire `if (x = 5)` au lieu de `if (x === 5)` est l'une des erreurs les plus fréquentes des débutants.

3.4 Les opérateurs logiques : `&&`, `||`, `!`

Les comparaisons seules ne suffisent pas : pour être admis au Probatoire, il faut une moyenne ≥ 10 et une conduite correcte ; pour payer en ligne, on utilise Orange Money ou MTN MoMo. Les opérateurs logiques combinent plusieurs conditions booléennes.

PROPRIÉTÉ Opérateurs logiques

- `a && b` (ET logique) : vaut `true` seulement si `a` et `b` sont vrais ;
- `a || b` (OU logique) : vaut `true` si `a` ou `b` (au moins l'un des deux) est vrai ;
- `!a` (NON logique) : inverse la valeur, `!true` vaut `false`.

EXEMPLE Exemple chiffré complet

```
let x = 5;
x += 3;           // x = 8
let y = 2 ** 3;   // y = 8
let z = (5 > 3) && (2 < 1); // true && false -> z = false
let admis = (x >= 10) || (y === 8); // false || true -> true
let mineur = !(x >= 18); // !(false) -> true
```

PROPRIÉTÉ Théorème (admis) : évaluation paresseuse ou court-circuit

JavaScript évalue les expressions logiques de gauche à droite et s'arrête dès que le résultat est connu :

- dans `a && b`, si `a` est falsy (faux, 0, "", null, undefined, NaN), alors `b` n'est pas évalué et le résultat est `a` ;
- dans `a || b`, si `a` est truthy, alors `b` n'est pas évalué et le résultat est `a`.

Ce comportement, appelé **court-circuit**, est très utile : `age && age >= 18` évite de tester une valeur absente.

LESAVAIS-TU ? L'opérateur ?? (nullish coalescing)

Depuis la norme ES2020, JavaScript propose l'opérateur `??` : `a ?? b` renvoie `b` uniquement si `a` vaut `null` ou `undefined` ; sinon il renvoie `a`. Contrairement à `||`, il ne déclenche pas le remplacement pour 0 ou "", qui sont des valeurs légitimes. Exemple : `let nom = saisie ?? "Visiteur"` ; affiche « Visiteur » seulement si rien n'a été saisi.

3.5 L'expression ternaire

Pour choisir entre deux valeurs selon une condition, JavaScript offre une écriture compacte en une seule ligne.

PROPRIÉTÉ Opérateur ternaire

La syntaxe `condition ? valeurSiVrai : valeurSiFaux` évalue la condition; si elle est vraie, l'expression vaut `valeurSiVrai`, sinon elle vaut `valeurSiFaux`. C'est l'équivalent raccourci d'un `if...else` qui affecte une variable.

EXEMPLE Mention automatique

```
let moyenne = 12.5;
let mention = (moyenne >= 10) ? "Admis" : "Ajourné";
// mention vaut "Admis"
let parite = (7 % 2 === 0) ? "pair" : "impair"; // "impair"
```

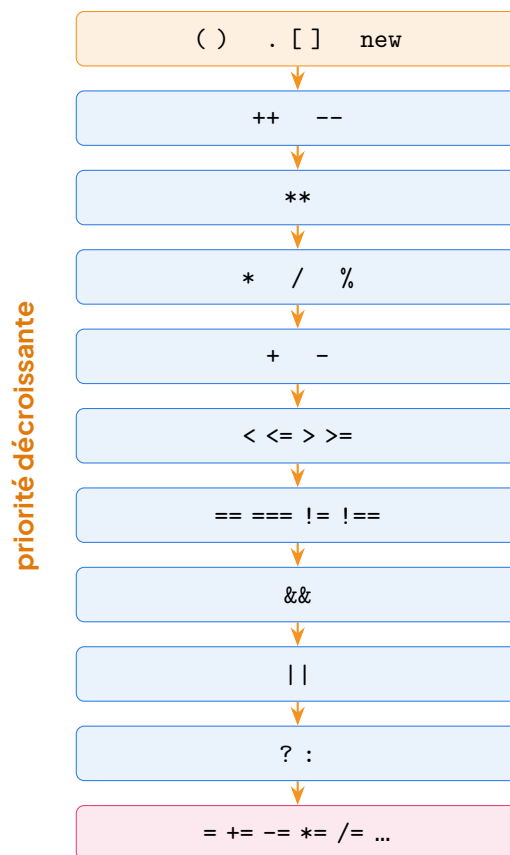
3.6 Priorité des opérateurs

Que vaut $2 + 3 * 4$? Comme en mathématiques, JavaScript ne lit pas simplement de gauche à droite : chaque opérateur possède un niveau de priorité. L'expression vaut $2 + (3 \times 4) = 14$, car $*$ est prioritaire sur $+$.

PROPRIÉTÉ Priorité des opérateurs (de la plus forte à la plus faible)

1. Groupement `( )`; accès membre `.` et `[ ]`, `new`;
2. Incrémentation `++`, `--`;
3. Puissance `**` (associative à droite);
4. Multiplication `*`, division `/`, modulo `%`;
5. Addition `+`, soustraction `-`;
6. Comparaisons `<`, `<=`, `>`, `>=`;
7. Égalités `==`, `===`, `!=`, `!==`;
8. ET logique `&&`;
9. OU logique `||`;
10. Ternaire `? :`;
11. Affectations `=`, `+=`, `-=`, `*=`, `/=` ...

En résumé : `**` avant `*` / avant `+` `-`; `&&` avant `||`; l'affectation arrive en dernier.



Pyramide de priorité des opérateurs JavaScript : les opérateurs du haut sont évalués avant ceux du bas.

MÉTHODE Évaluer une expression complexe pas à pas

Pour déterminer la valeur d'une expression sans se tromper :

1. Repère tous les opérateurs et classe-les par priorité ;
2. Évalue d'abord les parenthèses, puis les opérateurs de plus forte priorité ;
3. Remplace chaque sous-expression par sa valeur au fur et à mesure, en réécrivant l'expression simplifiée ;
4. Termine par l'affectation, toujours évaluée en dernier.

En cas de doute, parenthèse systématiquement pour lever toute ambiguïté : écris `(a + b) * c` plutôt que `a + b * c` si tu veux additionner d'abord. Les parenthèses ne coûtent rien et rendent le code plus lisible.

EXERCICE RÉSOLU 3 Évaluation complète d'une expression

Énoncé. Soit le programme suivant :

```
let a = 4, b = 3, c = 2;  
let r = a + b * c ** 2 > 10 && a !== b ? "OUI" : "NON";
```

Déterminer la valeur de `r` en détaillant chaque étape.

SOLUTION

Solution détaillée. On applique les priorités de la plus forte à la plus faible.

1. Puissance : $c ** 2 = 2^2 = 4$. L'expression devient `a + b * 4 > 10 && a !== b ? "OUI" : "NON"`.
2. Multiplication : $b * 4 = 3 \times 4 = 12$. Il reste `a + 12 > 10 && a !== b ? ....`
3. Addition : $a + 12 = 4 + 12 = 16$. Il reste `16 > 10 && a !== b ? ....`
4. Comparaison : `16 > 10` vaut `true`. Il reste `true && a !== b ? ....`
5. Égalité stricte : `a !== b`, c'est-à-dire `4 !== 3`, vaut `true`. Il reste `true && true ? ....`
6. ET logique : `true && true` vaut `true`. Il reste `true ? "OUI" : "NON"`.
7. Ternaire : la condition est vraie, donc l'expression vaut `"OUI"`.
8. Affectation (en dernier) : `r` reçoit la valeur `"OUI"`.

Conclusion : `r = "OUI"`. Remarque l'ordre : la puissance avant la multiplication, la multiplication avant l'addition, les comparaisons avant `&&`, et l'affectation tout à la fin.

EXPÉRIENCE TP : tester les opérateurs dans la console du navigateur

Objectif : vérifier expérimentalement les priorités et le comportement des opérateurs.
Matériel : un ordinateur avec un navigateur (Chrome ou Firefox).

1. Ouvre un nouvel onglet, puis la console avec la touche F12 (onglet Console).
2. Tape `2 + 3 * 4` puis Entrée : observe 14. Tape ensuite `(2 + 3) * 4` : observe 20.
3. Teste la coercition : `5 == "5"` donne `true`, mais `5 === "5"` donne `false`.
4. Teste le court-circuit : `false && alert("coucou")` n'affiche rien, alors que `true && alert("coucou")` ouvre la boîte de dialogue.
5. Teste le ternaire : `let m = 8; m >= 10 ? "Admis" : "Ajourné"` affiche "Ajourné".

Interprétation : la console évalue chaque expression en respectant la pyramide de priorité ; le court-circuit empêche l'exécution de la seconde partie lorsque le résultat est déjà connu.

À RETENIR L'essentiel de la section

- Les opérateurs arithmétiques `+` `-` `*` `/` construisent les calculs ; `%` donne le reste et `Math.pow` la puissance.
- `x += a` équivaut à `x = x + a` ; `++` et `--` ajoutent ou retirent 1 (préfixe : nouvelle valeur ; postfixe : ancienne valeur).
- Privilégie `===` et `!==` : `==` convertit les types et réserve de mauvaises surprises.
- `&&`, `||` et `!` combinent les conditions, avec évaluation en court-circuit.
- Le ternaire `cond ? v1 : v2` remplace un `if...else` d'affectation.
- Priorité : `**` avant `*` / avant `+` `-` ; `&&` avant `||` ; l'affectation en dernier. En cas de doute : parenthèse !

Ces opérateurs sont les briques élémentaires de tout calcul. Dans la section suivante, nous les assemblerons dans les structures de contrôle — instructions séquentielles, conditionnelles et itératives — pour donner à nos programmes la capacité de décider et de répéter.

4 STRUCTURES DE CONTRÔLE : SÉQUENTIELLES, CONDITIONNELLES, ITÉRATIVES

Dans la section précédente, nous avons appris à construire des **expressions** à l'aide des opérateurs arithmétiques, de comparaison et logiques : nous savons désormais calculer une valeur, tester une égalité ou combiner plusieurs conditions. Mais une expression seule ne fait rien d'elle-même. Pour obtenir un véritable programme, il faut organiser les instructions dans le temps : les exécuter dans l'ordre, n'en exécuter certaines que si une condition est vraie, ou les répéter tant qu'un objectif n'est pas atteint. C'est exactement le rôle des **structures de contrôle**. Tu les as déjà rencontrées en algorithmique sous forme de pseudo-code (`Si...Alors`, `Pour...FinPour`, `Tant que...`) ; nous allons maintenant les traduire en JavaScript, presque mot à mot.

4.1 La structure séquentielle : l'ordre naturel des instructions

DÉFINITION Instruction et séquence

Une **instruction** est un ordre élémentaire donné à l'ordinateur (affectation, affichage, appel de fonction). Un programme JavaScript est par défaut une **structure séquentielle** : les instructions s'exécutent les unes après les autres, de haut en bas, dans l'ordre d'écriture. Chaque instruction se termine par un **point-virgule** ; (celui-ci est optionnel en fin de ligne, mais son usage systématique est fortement recommandé).

EXEMPLE Une séquence simple

```
let prix = 5000;           // instruction 1 : declaration
let quantite = 3;          // instruction 2
let total = prix * quantite; // instruction 3 : utilise les deux precedentes
alert("Total : " + total + " FCFA"); // instruction 4 : affiche 15000
```

L'ordre compte : si l'on affichait `total` avant de l'avoir calculé, le résultat serait `undefined`. L'ordinateur est un exécutant discipliné mais sans imagination : il fait exactement ce qui est écrit, dans l'ordre où c'est écrit.

ATTENTION Le point-virgule oublié

JavaScript tolère l'absence de `;` en fin de ligne, mais certains cas ambigus provoquent des erreurs difficiles à diagnostiquer. Règle d'or au lycée : terminer systématiquement chaque instruction par un point-virgule.

4.2 Les structures conditionnelles : `if`, `else if`, `else`

Intuition. Le bureau d'un lycée affiche « Admis » si la moyenne est au moins 10, « Ajourné » sinon. Le programme doit choisir entre plusieurs chemins : c'est la structure conditionnelle.

PROPRIÉTÉ Syntaxe de la conditionnelle

```
if (condition1){
    // instructions executees si condition1 est vraie
} else if (condition2){
    // instructions si condition1 fausse ET condition2 vraie
} else {
    // instructions si aucune condition n'est vraie
}
```

La `condition` est une expression booléenne (qui vaut `true` ou `false`). Les accolades `{` et `}` délimitent un **bloc** d'instructions. Les parties `else if` (répétables) et `else` (au plus une) sont facultatives.

EXEMPLE Appréciation d'une note

```
let moyenne = parseFloat(prompt("Moyenne generale ?"));
if (moyenne >= 16){
```

```
    alert("Excellent !");
} else if (moyenne >= 14) {
    alert("Bien");
} else if (moyenne >= 10) {
    alert("Passable : admis");
} else {
    alert("Insuffisant : ajourne");
}
```

Les conditions sont évaluées dans l'ordre : dès que l'une est vraie, son bloc s'exécute et les suivantes sont ignorées. Ainsi, pour une moyenne de 15, le test `moyenne >= 16` échoue, puis `moyenne >= 14` réussit : seul « Bien » est affiché.

ATTENTION Confondre = et ===

Écrire `if (note = 10)` au lieu de `if (note === 10)` ne teste rien : cela affecte 10 à `note` et la condition est alors considérée comme vraie ! En JavaScript, la comparaison stricte s'écrit `===` (égalité de valeur et de type). C'est l'erreur classique n°1 des débutants.

4.3 Le choix multiple : switch / case / default

Quand une même variable doit être comparée à plusieurs valeurs précises (un jour de la semaine, un code de région, une option de menu), enchaîner les `else if` devient lourd. La structure **switch** est plus lisible.

PROPRIÉTÉ Syntaxe du switch

```
switch (expression) {
    case valeur1 :
        // instructions si expression === valeur1
        break;
    case valeur2 :
        // instructions si expression === valeur2
        break;
    default :
        // instructions si aucun cas ne correspond
}
```

La comparaison est stricte (`===`) : valeur et type doivent coïncider. Le mot-clé `break` est indispensable pour sortir du `switch` ; sans lui, l'exécution « tombe » dans le cas suivant (fall-through).

EXEMPLE Menu d'un convertisseur

```
let choix = parseInt(prompt("1 : FCFA->Euro 2 : Euro->FCFA"));
switch (choix) {
  case 1:
    alert("Conversion FCFA vers Euro");
    break;
  case 2:
    alert("Conversion Euro vers FCFA");
    break;
  default:
    alert("Choix invalide !");
}
```

Ici `parseInt` est crucial : `prompt` renvoie une chaîne de caractères, et `"1" === 1` est faux (comparaison stricte). Sans conversion, c'est le `default` qui s'exécuterait toujours.

4.4 Les boucles : répéter sans se répéter

Intuition. Un cybercafé de Yaoundé veut imprimer 30 factures numérotées. Écrire 30 fois la même instruction serait absurde : on écrit l'instruction une seule fois dans une **boucle**, et l'ordinateur la répète autant que nécessaire. Il existe trois formes de boucles, qui correspondent exactement aux structures vues en algorithmique.

La boucle `for` : nombre d'itérations connu.

PROPRIÉTÉ Syntaxe de `for`

```
for ( initialisation ; condition; incrementation) {
  // corps de la boucle
}
```

1. L'`initialisation` s'exécute une seule fois au départ ;
2. la `condition` est testée avant chaque tour : si elle est fausse, la boucle s'arrête ;
3. le `corps` s'exécute, puis l'`incrément` s'exécute, et on retourne à l'étape 2.

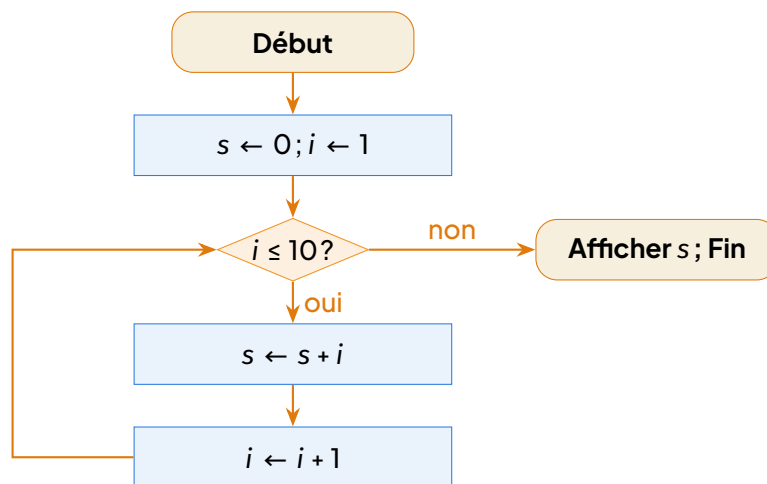
C'est la boucle typique lorsqu'on sait combien de fois on veut répéter.

EXEMPLE Somme des 10 premiers entiers

```
// Somme 1..10
let s = 0;
```

```
for (let i = 1; i <= 10; i++){  
  s += i; // equivaut a s = s + i  
}  
console.log(s); // 55
```

Trace d'exécution : i vaut successivement 1, 2, ..., 10 ; à chaque tour s accumule la valeur de i . Quand i devient 11, la condition $i \leq 10$ est fausse et la boucle s'arrête. Résultat : $1 + 2 + \dots + 10 = 55$.



Organigramme de la boucle `for` calculant $s = 1 + 2 + \dots + 10$: tant que $i \leq 10$, on accumule puis on incrémente.

La boucle `while` : répéter tant qu'une condition est vraie.

PROPRIÉTÉ Syntaxe de `while`

```
while (condition){  
  // corps : execute tant que condition est vraie  
}
```

Le test a lieu avant chaque itération : si la condition est fausse dès le départ, le corps ne s'exécute jamais. On l'utilise quand le nombre de tours n'est pas connu à l'avance (saisie jusqu'à une valeur valide, division répétée, etc.).

EXEMPLE Saisie contrôlée d'une note

```
let note = parseFloat(prompt("Note (0 à 20) ?"));
while (isNaN(note) || note < 0 || note > 20) {
    alert("Valeur incorrecte !");
    note = parseFloat(prompt("Note (0 à 20) ?"));
}
alert("Note enregistrée : " + note);
```

On ne sait pas combien de fois l'utilisateur se trompera : le `while` est le bon outil. La fonction `isNaN` détecte le cas où la saisie n'est pas un nombre (par exemple si l'utilisateur tape « bonjour »).

La boucle `do...while` : au moins une exécution.

PROPRIÉTÉ Syntaxe de `do...while`

```
do {
    // corps : execute AU MOINS une fois
} while (condition);
```

Le test a lieu après le corps : celui-ci s'exécute donc toujours au moins une fois. Idéal pour un menu que l'on affiche d'abord, puis que l'on réaffiche tant que l'utilisateur ne quitte pas. Attention au `;` final après la parenthèse !

EXEMPLE Menu répété

```
let rep;
do {
    rep = prompt("Convertir encore ? (o/n)");
} while (rep !== "o" && rep !== "n");
```

La question est posée une première fois, puis reposée tant que la réponse n'est ni "o" ni "n".

Interrompre une boucle : `break` et `continue`.

DÉFINITION `break` et `continue`

- **break** : sort immédiatement de la boucle (ou du `switch`); l'exécution reprend après l'accolade fermante ;
- **continue** : abandonne le tour en cours et passe directement à l'itération suivante (le

test et l'incrémentation sont effectués).

EXEMPLE Recherche et filtrage

```
// Cherche le premier multiple de 7 supérieur à 50
for (let n = 51; ; n++){
  if (n % 7 === 0) { alert("Trouve : " + n); break; } // 56
}

// Affiche les entiers de 1 à 10 sauf les multiples de 3
for (let i = 1; i <= 10; i++){
  if (i % 3 === 0) continue; // on saute 3, 6, 9
  console.log(i);
}
```

Dans la première boucle, la condition est volontairement vide (toujours vraie) : c'est le `break` qui assure la sortie dès que l'on trouve 56.

ATTENTION La boucle infinie

Oublier l'incrémentation dans un `while` (`while (i <= 10) { s += i; }` sans `i++`) ou écrire une condition toujours vraie (`for (let i = 1; i >= 1; i++)`) produit une **boucle infinie** : la page se fige et le navigateur finit par proposer d'arrêter le script. Règle de survie : avant de lancer une boucle, vérifie toujours (1) que la variable de contrôle est initialisée, (2) qu'elle évolue dans le corps, (3) que la condition finira par devenir fausse.

PROPRIÉTÉ Portée de `let` dans les boucles

Une variable déclarée avec `let` dans l'en-tête d'un `for` (`for (let i = ...)`) est locale à la boucle : elle n'existe pas après l'accolade fermante. De plus, `let` crée une nouvelle liaison à chaque itération : si le corps définit une fonction qui utilise `i`, chaque fonction « se souvient » de la valeur de `i` à son tour (mécanisme de fermeture). Avec l'ancien mot-clé `var`, une seule variable était partagée, source de bugs célèbres. Au lycée comme en pratique professionnelle : toujours utiliser `let` (ou `const` si la variable n'est pas réaffectée).

PROPRIÉTÉ Invariant de boucle (théorème admis)

Un **invariant de boucle** est une propriété qui reste vraie avant et après chaque itération. Pour la somme des 10 premiers entiers, l'invariant est : « au début du tour numéro k , $s =$

$1 + 2 + \dots + (k - 1)$ ». Vrai au départ ($s = 0$), conservé à chaque tour, il garantit qu'à la sortie $s = 1 + 2 + \dots + 10 = 55$: la boucle est correcte. Cette notion, fondamentale pour prouver les programmes, est ici admise et non exigée au Probatoire.

4.5 Traduire un algorithme en JavaScript

MÉTHODE De l'algorithme au code JavaScript

1. Identifier les variables : pour chaque Variable x : ... du pseudo-code, écrire `let x;` (avec `prompt + parseInt/parseFloat` pour les entrées).
2. Repérer les conditions : Si...Alors...Sinon...FinSi devient `if / else` avec accolades.
3. Repérer les boucles : Pour i de 1 à n devient `for (let i = 1; i <= n; i++);` Tant que... devient `while`.
4. Traduire les sorties : Écrire(...) devient `alert(...)`, `document.write(...)` ou `console.log(...)`.
5. Tester mentalement sur un petit exemple (table de trace) avant d'exécuter.

EXERCICE RÉSOLU 4 Table de multiplication d'un nombre

Énoncé : traduis en JavaScript l'algorithme suivant, qui affiche la table de multiplication d'un entier n saisi par l'utilisateur.

```
Algorithme TableMultiplication
Variable n, i : Entier
Debut
    Ecrire("Entrez un entier : ")
    Lire(n)
    Pour i de 1 à 10 Faire
        Ecrire(n, " x ", i, " = ", n*i)
    FinPour
Fin
```

SOLUTION

Solution détaillée : la variable `n` vient d'une saisie, donc `prompt` suivi de `parseInt` (étape 1); la boucle `Pour` devient un `for` de 1 à 10 inclus, d'où la condition `i <= 10` (étape 3); l'écriture devient `document.write` avec concaténation (étape 4).

```
let n = parseInt(prompt("Entrez un entier :"));
for (let i = 1; i <= 10; i++){
  document.write(n + " x " + i + " = " + (n * i) + "<br>");
}
```

Vérification pour $n = 7$: le premier tour affiche $7 \times 1 = 7$, le dernier $7 \times 10 = 70$, puis i vaut 11 et la boucle s'arrête. La correspondance ligne à ligne avec le pseudo-code est immédiate : c'est toute la force de l'algorithmique, qui rend le langage presque transparent.

LESAVAIS-TU ? La boucle `for...of` (ES6)

Depuis la norme ECMAScript 2015 (ES6), JavaScript propose `for...of`, qui parcourt directement les valeurs d'un objet itérable (tableau, chaîne de caractères, `Map`) sans gérer d'indice :

```
let prenom = ["Amina", "Biya", "Etoundi"];
for (let p of prenom) {
  console.log(p); // affiche chaque prenom, sans i !
}
```

Plus courte et moins sujette aux erreurs d'indices, elle coexiste avec le `for` classique, qui reste indispensable dès qu'on a besoin de la position ou d'un pas particulier.

À RETENIR L'essentiel de la section

- Par défaut, les instructions s'exécutent séquentiellement, chacune terminée par `;`.
- `if` / `else if` / `else` choisit un chemin; `switch` compare strictement (`===`) une expression à plusieurs valeurs, avec `break` pour sortir.
- `for` : nombre de tours connu; `while` : test avant, zéro exécution possible; `do...while` : test après, au moins une exécution.
- `break` sort de la boucle, `continue` saute au tour suivant.
- Toute boucle doit avoir une condition de sortie atteignable : méfie-toi des boucles infinies !

EXERCICE 3 À toi de jouer

1. Écris un script qui demande un entier n et affiche « pair » ou « impair » (opérateur `%`).
2. Écris une boucle `while` qui calcule le plus petit entier k tel que $2^k \geq 1000$.

3. Traduis en JavaScript : « Saisir l'âge ; si âge < 18 afficher "mineur", sinon si âge ≤ 60 afficher "adulte", sinon "senior" ».

CORRIGÉ Exercice n°1

```
let n = parseInt(prompt("Entrez un entier :"));
if (n % 2 === 0) {
  alert(n + " est pair");
} else {
  alert(n + " est impair");
}
```

CORRIGÉ Exercice n°2

```
let k = 0, p = 1;      // p = 2^k
while (p < 1000) {
  p = p * 2;          // on multiplie par 2
  k++;                // et on compte
}
alert("k = " + k);     // k = 10 car 2^10 = 1024 >= 1000
```

Invariant : au début de chaque tour, $p = 2^k$. La boucle s'arrête dès que $p \geq 1000$, donc k est bien le plus petit entier cherché.

CORRIGÉ Exercice n°3

```
let age = parseInt(prompt("Votre age ?"));
if (age < 18) {
  alert("mineur");
} else if (age <= 60) {
  alert("adulte");
} else {
  alert("senior");
}
```

5 FONCTIONS ET GESTION D'ÉVÉNEMENTS (ONCLICK)

Dans la section précédente, tu as appris à traduire en JavaScript les structures de contrôle : séquences d'instructions, alternatives (`if/else`) et boucles (`while, for`). Tu peux donc déjà écrire des programmes complets. Mais imagine la situation suivante : dans le site de la boutique de ta famille, tu dois calculer la TVA de 19,25 % à dix endroits différents de la page. Vas-tu recopier dix fois le même calcul ? Et si le taux change, vas-tu modifier dix lignes ? C'est exactement le problème que résolvent les **fonctions** : écrire une fois un bloc d'instructions, lui donner un nom, et le réutiliser autant de fois que nécessaire. Mieux encore : grâce aux **événements** comme le clic de souris, ces fonctions pourront être déclenchées par l'utilisateur, rendant enfin tes pages interactives.

5.1 Déclaration et appel d'une fonction

DÉFINITION Fonction

Une **fonction** est un sous-programme réutilisable, identifié par un nom, qui regroupe un bloc d'instructions. Elle peut recevoir des données en entrée appelées **paramètres**, et peut renvoyer un résultat grâce au mot-clé `return`. Tant qu'elle n'est pas appelée, une fonction ne s'exécute pas : sa déclaration ne fait que la mémoriser.

PROPRIÉTÉ Syntaxe de déclaration d'une fonction

```
function nomDeLaFonction(parametre1, parametre2) {  
    // instructions  
    return resultat; // facultatif  
}
```

Règles à respecter :

- le mot-clé `function` est obligatoire, suivi du nom de la fonction (mêmes règles que pour les variables : lettres, chiffres, `_`, sans commencer par un chiffre);
- les parenthèses sont obligatoires, même sans paramètre : `function saluer() { ... }`;
- les accolades délimitent le corps de la fonction;
- `return` renvoie une valeur et arrête immédiatement l'exécution de la fonction : toute instruction placée après un `return` exécuté est ignorée;
- sans `return`, la fonction renvoie la valeur spéciale `undefined`.

EXEMPLE Une fonction qui calcule un carré

```
// Declaration de la fonction
function carre(x) {
    return x * x;
}

// Appels de la fonction
alert(carre(7)); // affiche 49
let aire = carre(12); // aire vaut 144
alert(carre(2) + carre(3)); // affiche 13
```

L'appel consiste à écrire le nom suivi des **arguments** entre parenthèses : `carre(7)`. La valeur 7 est copiée dans le paramètre `x`, le corps s'exécute, et le résultat 49 remplace l'appel dans l'expression.

ATTENTION Paramètre et argument : ne pas confondre

Le **paramètre** est la variable déclarée dans l'en-tête (`x` dans `function carre(x)`). L'**argument** est la valeur concrète fournie lors de l'appel (7 dans `carre(7)`). Autre piège classique : écrire `carre` sans parenthèses ne lance pas la fonction, cela désigne la fonction elle-même comme une valeur.

PROPRIÉTÉ Hoisting (remontée des déclarations)

Les déclarations de fonctions avec le mot-clé `function` sont remontées (hoisted) en haut de leur portée par le moteur JavaScript : on peut donc appeler une fonction avant sa déclaration dans le code. En revanche, les expressions de fonction affectées à `let` ou `const` sont remontées mais restent dans la zone morte temporelle (TDZ) jusqu'à leur initialisation : les appeler avant leur définition provoque une erreur `ReferenceError`.

```
direBonjour(); // OK : declaration remontee
function direBonjour() { alert("Bonjour !"); }

doubler(5); // ERREUR ReferenceError : const en TDZ ici
const doubler = function(x) { return x * 2; };
```

5.2 Expressions de fonction et fonctions fléchées

LESAVAIS-TU ? Les fonctions sont des valeurs de première classe

En JavaScript, une fonction est une valeur comme une autre : on peut l'affecter à une variable, la passer en argument à une autre fonction, et même la retourner comme résultat. On dit que les fonctions sont des « objets de première classe ». C'est l'une des grandes forces du langage, et c'est ce qui rend possible la gestion des événements : on donne une fonction au navigateur pour qu'il l'exécute plus tard, au moment du clic.

DÉFINITION Fonction anonyme et expression de fonction

Une **fonction anonyme** est une fonction sans nom. Une **expression de fonction** consiste à créer une fonction (souvent anonyme) et à l'affecter à une variable :

```
const f = function(x) {  
  return x * 2;  
};  
alert(f(21)); // affiche 42
```

Depuis la norme ES6 (2015), on dispose d'une écriture plus courte : la **fonction fléchée** (arrow function) :

```
const double = x => x * 2; // un seul parametre, return implicite  
const somme = (a, b) => a + b; // plusieurs parametres : parentheses  
const saluer = () => alert("Salut !"); // aucun parametre : parentheses vides  
  
// Forme avec bloc : le return redevient obligatoire  
const maxi = (a, b) => {  
  if (a > b) { return a; }  
  return b;  
};
```

Si le corps tient en une seule expression, on omet les accolades et le mot `return` (retour implicite).

5.3 Portée des variables et passage des arguments

DÉFINITION Portée locale

Les paramètres et les variables déclarées avec `let` ou `const` à l'intérieur d'une fonction sont **locales** : elles n'existent que pendant l'exécution de la fonction et sont invisibles à l'extérieur. Une variable déclarée hors de toute fonction est **globale** : accessible partout. Une fonction peut lire une variable globale ; on parle de **fermeture** (closure) lorsqu'une fonction « se souvient » des variables de l'environnement où elle a été créée, même après la fin de cet environnement.

EXEMPLE Portée locale et closure

```
function compteur() {  
  let n = 0;           // variable locale  
  return function() {  // fonction anonyme retournée  
    n = n + 1;         // elle "se souvient" de n : closure  
    return n;  
  };  
}  
  
const clic = compteur();  
alert(clic()); // 1  
alert(clic()); // 2 : n a été conservé entre les appels
```

PROPRIÉTÉ Passage par valeur et par référence

- Les types primitifs (nombres, chaînes, booléens) sont passés par valeur : la fonction reçoit une copie ; modifier le paramètre ne modifie pas la variable de l'appelant.
- Les objets (tableaux, objets) sont passés par référence (plus précisément : la référence est passée par valeur) : la fonction reçoit l'adresse de l'objet ; modifier son contenu à l'intérieur de la fonction modifie l'objet original.

```
function essai(nombre, tableau) {  
  nombre = 99;      // ne modifie que la copie locale  
  tableau[0] = 99;  // modifie le tableau original !  
}  
  
let a = 5;  
let t = [1, 2, 3];  
essai(a, t);  
alert(a); // 5 : inchangé (passage par valeur)  
alert(t[0]); // 99 : modifié (passage par référence)
```

5.4 Gestion des événements : onclick et addEventListener

Voici enfin la clé de l'interactivité. Jusqu'ici, ton code s'exécutait au chargement de la page. Avec les **événements**, le navigateur surveille les actions de l'utilisateur (clic, saisie, survol...) et exécute la fonction que tu as associée à chaque action.

DÉFINITION Événement et gestionnaire d'événement

Un **événement** est un signal émis par le navigateur lorsqu'une action se produit sur la page

(clic, frappe au clavier, chargement terminé...). Un **gestionnaire d'événement** (listener) est la fonction que l'on associe à cet événement pour y répondre. L'événement de clic se nomme `click`; l'attribut HTML correspondant est `onclick`.

Il existe deux façons d'associer une fonction à un clic :

```
<!-- Methode 1 : attribut HTML (simple mais deconseillee) -->
<button onclick="convertir()">Convertir</button>

<!-- Methode 2 : addEventListener (bonne pratique) -->
<button id="btn">Convertir</button>
<script>
    function convertir() {
        alert("Conversion en cours...");
    }
    document.getElementById('btn').addEventListener('click', convertir);
</script>
```

MÉTHODE Associer proprement une fonction à un bouton

1. Dans le HTML, donner un `id` unique au bouton : `<button id="btn">`.
2. Dans le script, définir la fonction de traitement, par exemple `function convertir() { ... }`.
3. Récupérer l'élément : `document.getElementById('btn')`.
4. Brancher l'écouteur : `.addEventListener('click', convertir);` — attention, on écrit `convertir` sans parenthèses : on transmet la fonction, on ne l'appelle pas.
5. Placer le script en fin de `<body>` pour que le bouton existe déjà au moment du branchement.

ATTENTION `convertir` ou `convertir()` ?

Écrire `addEventListener('click', convertir())` appelle la fonction immédiatement au chargement et branche son résultat (souvent `undefined`) : le clic ne fera plus rien. Il faut passer la référence de la fonction : `addEventListener('click', convertir)`.

ATTENTION Le mot-clé `this` dans les gestionnaires

Dans un gestionnaire écrit avec `function` classique, `this` désigne l'élément HTML cible

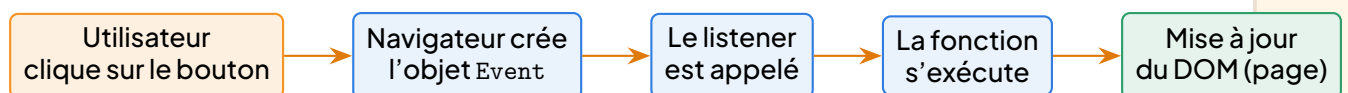
du clic (le bouton). Avec une fonction fléchée, `this` désigne le contexte lexical englobant (souvent `window`), donc pas le bouton. Pour modifier le bouton cliqué, préfère une fonction classique ou utilise `event.target`.

DÉFINITION L'objet `event`

Lors d'un événement, le navigateur crée un objet `event` (souvent noté `e`) qu'il transmet au gestionnaire. Propriétés utiles :

- `e.target` : l'élément HTML qui a déclenché l'événement ;
- `e.type` : le nom de l'événement ("click") ;
- `e.preventDefault()` : annule le comportement par défaut (par exemple l'envoi d'un formulaire ou la navigation d'un lien).

```
document.getElementById('btn').addEventListener('click', function(e) {  
    alert("Clic sur : " + e.target.id + " (evenement " + e.type + ")");  
});
```



Flux d'un événement de clic : de l'action de l'utilisateur à la mise à jour de la page.

PROPRIÉTÉ Bonne pratique : séparer HTML et JavaScript

Préfère `addEventListener` à l'attribut `onclick` : le code HTML décrit la structure, le code JavaScript décrit le comportement. Cette séparation rend le site plus lisible, plus facile à maintenir, et évite de multiplier les fonctions globales inutiles. Regroupe tes scripts en fin de `<body>` ou dans un fichier `.js` externe lié par `<script src="script.js"></script>`.

EXERCICE RÉSOLU 5 Un bouton qui calcule la TVA

Énoncé : écrire une page contenant un bouton « Calculer la TVA ». Au clic, la page demande un prix hors taxe (en FCFA) via `prompt()`, calcule la TVA de 19,25 % à l'aide d'une fonction `tva(montant)`, puis affiche le montant de la TVA avec `alert()`.

SOLUTION

Solution détaillée :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Calcul de TVA</title>
</head>
<body>
  <button id="btnTva">Calculer la TVA</button>

  <script>
    // 1. Declaration de la fonction de calcul
    function tva(montant) {
      return montant * 0.1925;
    }

    // 2. Gestionnaire d'événement
    function calculer() {
      let saisie = prompt("Prix hors taxe (FCFA) :");
      // Attention : parseFloat attend un point decimal, pas une virgule
      let prix = parseFloat(saisie.replace(',', ' '));
      if (isNaN(prix)) {
        alert("Veuillez saisir un nombre valide.");
      } else {
        alert("TVA = " + tva(prix) + " FCFA");
      }
    }

    // 3. Branchement de l'événement click
    document.getElementById('btnTva').addEventListener('click', calculer);
  </script>
</body>
</html>
```

Trace d'exécution : l'utilisateur clique, saisit 100000; parseFloat donne 100000; tva(100000) renvoie 100000 × 0,1925 = 19250; la boîte affiche « TVA = 19250 FCFA ».

ATTENTION parseFloat et le séparateur décimal

En JavaScript, parseFloat (et parseInt) ne reconnaissent que le point comme séparateur décimal. Au Cameroun, on utilise la virgule (ex. 1500,50). Si l'utilisateur tape une virgule, parseFloat("1500,50") renverra 1500 (il s'arrête à la virgule). Solution : remplacer la virgule par un point avant conversion : parseFloat(saisie.replace(',', ' ')). Cette astuce sera indispensable pour vos convertisseurs de la section 6.

EXERCICE 4 À toi de jouer

1. Écris une fonction fléchée `cube` qui retourne le cube d'un nombre, puis teste-la avec `alert(cube(3))`.
2. Crée une page avec deux boutons partageant le même gestionnaire : au clic, affiche l'id du bouton cliqué grâce à `event.target`.
3. Que vaut `a` après l'exécution de `function f(x){x=0;} puis let a=8; f(a);` ? Justifie avec la notion de passage par valeur.

CORRIGÉ Exercice

1. `const cube = x => x * x * x; puis alert(cube(3));` affiche 27.
2. On écrit un seul gestionnaire `function qui(e){ alert(e.target.id); }` que l'on branche sur chaque bouton avec `addEventListener('click', qui)`.
3. `a` vaut toujours 8 : le nombre est passé par valeur, la fonction ne modifie que sa copie locale `x`.

À RETENIR L'essentiel de la section

Une fonction se déclare avec `function`, `const f = function(...)` ou la forme fléchée `const f = x => ....` Elle s'exécute uniquement lorsqu'on l'appelle. Les déclarations `function` sont remontées (hoisting), pas les expressions `const` (zone morte temporelle). Les variables locales sont invisibles hors de la fonction; les primitifs sont passés par valeur, les objets par référence. Enfin, l'interactivité repose sur les événements : `addEventListener('click', maFonction)` associe une fonction à un clic, et l'objet `event` renseigne sur l'action de l'utilisateur. Tu disposes maintenant de toutes les briques pour construire, dans la prochaine section, de véritables convertisseurs interactifs en maîtrisant `prompt()`, `alert()`, `confirm()`, `parseInt()`, `parseFloat()` et `document.write()`.

6 ENTRÉES/SORTIES ET RÉALISATION DE CONVERTISSEURS PRATIQUES

Après avoir maîtrisé la déclaration de fonctions et la gestion de l'événement `onclick` (section 5), nous disposons désormais de tous les outils pour construire des applications web interactives complètes. Une application, aussi simple soit-elle, suit invariablement le schéma **Entrée** → **Traitement** → **Sortie**. Cette section explore en profondeur les mécanismes natifs d'entrée/-

sortie de JavaScript, les bonnes pratiques de manipulation du DOM pour l’affichage, et aboutit à la conception rigoureuse de trois convertisseurs pratiques : devises, températures et dates.

6.1 Boîtes de dialogue natives : `prompt`, `alert`, `confirm`

Historiquement, JavaScript propose trois méthodes bloquantes (modales) appartenant à l’objet `window` (souvent omis) pour interagir avec l’utilisateur sans formulaire HTML. Elles sont utiles pour des scripts pédagogiques, des tests rapides ou des confirmations critiques, mais leur aspect visuel est imposé par le navigateur et non personnalisable.

DÉFINITION Boîtes de dialogue natives

- `prompt(message, default)` : Affiche une boîte avec un champ de saisie. Renvoie la chaîne de caractères saisie, ou `null` si l’utilisateur annule. Le second argument (optionnel) pré-remplit le champ.
- `alert(message)` : Affiche un message d’information avec un bouton « OK ». Ne renvoie aucune valeur (`undefined`). Bloque l’exécution du script jusqu’à la fermeture.
- `confirm(question)` : Affiche une boîte avec « OK » et « Annuler ». Renvoie un booléen : `true` si OK, `false` si Annuler.

EXEMPLE Utilisation basique des boîtes natives

```
// 1. Saisie d'un nom (chaîne)
let nom = prompt("Entrez votre nom :", "Visiteur");
if (nom !== null && nom.trim() !== "") {
    alert("Bienvenue " + nom + " !");
} else {
    alert("Aucun nom saisi.");
}

// 2. Confirmation avant action irréversible
let confirmation = confirm("Voulez-vous vraiment supprimer ce fichier ?");
if (confirmation) {
    // code de suppression...
    alert("Fichier supprimé.");
} else {
    alert("Suppression annulée.");
}
```

ATTENTION Pièges de `prompt` et conversions

`prompt` renvoie toujours une chaîne (string), même si l'utilisateur tape des chiffres. Toute opération arithmétique (+, -, *, /) sur cette chaîne sans conversion préalable provoquera une concaténation (pour +) ou une conversion implicite hasardeuse (pour -, *, /).

```
let a = prompt("Nombre A :"); // ex. "10"
let b = prompt("Nombre B :"); // ex. "5"
alert(a + b); // Affiche "105" (concaténation !) et non 15.
```

Solution : Convertir explicitement avec `Number()`, `parseInt()` ou `parseFloat()` (voir 6.2).

6.2 Écriture dans le document : `document.write` vs manipulation du DOM

PROPRIÉTÉ `document.write(texte)`

Écrit directement dans le flux HTML pendant le chargement initial de la page (parsing). Si elle est appelée après le chargement complet (ex. dans un gestionnaire d'événement `onclick`), elle écrase intégralement le document (page blanche) et y écrit le nouveau contenu. À proscrire dans les applications interactives modernes.

À RETENIR Bonnes pratiques d'affichage (DOM)

Pour modifier le contenu d'une page déjà chargée, on sélectionne un élément HTML (via `id`, `class`, `tag`) et on modifie ses propriétés :

- `element.textContent = "texte"` : Insère du texte pur (échappe les balises HTML, sécurisé contre XSS). Préféré par défaut.
- `element.innerHTML = "<b>HTML</b>"` : Interprète la chaîne comme du HTML. Utile pour injecter une structure complexe, mais dangereux si la chaîne provient d'une saisie utilisateur non validée (failles XSS).
- `element.value = "valeur"` : Modifie la valeur d'un champ de formulaire (`<input>`, `<select>`, `<textarea>`).

EXEMPLE Manipulation moderne du DOM (à privilégier)

```
<!-- HTML -->
<p id="message">Contenu initial</p>
<input type="text" id="champSaisie" placeholder="Tapez ici">
<button onclick="mettreAJour()">Mettre à jour</button>
```

```
<script>
function mettreAJour() {
    // 1. Récupération des éléments
    let zoneAffichage = document.getElementById("message");
    let champ = document.getElementById("champSaisie");

    // 2. Lecture de la valeur saisie (ENTRÉE)
    let saisie = champ.value; // Toujours une chaîne

    // 3. Traitement (ex : mise en majuscules)
    let resultat = saisie.toUpperCase();

    // 4. Affichage (SORTIE) – Sécurisé
    zoneAffichage.textContent = "Vous avez tapé : " + resultat;

    // 5. Bonus : vider le champ après action
    champ.value = "";
}
</script>
```

6.3 Conversions de type : Number, parseInt, parseFloat

Le passage de string (saisie) à number (calcul) est l'opération la plus critique en JavaScript.

PROPRIÉTÉ Fonctions de conversion vers nombre

Fonction	Comportement	Renvoie
Number(chaine)	Conversion stricte. Espaces ignorés. Échoue si format invalide.	number ou NaN
parseInt(chaine, base)	Analyse depuis le début, s'arrête au 1er caractère non numérique. Base 10 par défaut (mais toujours préciser la base 10).	integer ou NaN
parseFloat(chaine)	Analyse depuis le début, accepte un point décimal, s'arrête au 1er caractère invalide. Ignore espaces initiaux.	float ou NaN

PROPRIÉTÉ Cas particuliers de `parseFloat`

- `parseFloat(" 12.34 ")` → 12.34 (espaces ignorés).
- `parseFloat("12.34.56")` → 12.34 (s'arrête au 2e point).
- `parseFloat("12px")` → 12 (utile pour CSS).
- `parseFloat("abc")` → NaN.

EXEMPLE Conversion robuste pour un convertisseur

```
function lireNombre(idChamp) {  
    let champ = document.getElementById(idChamp);  
    let brut = champ.value.trim(); // Supprime espaces début/fin  
  
    if (brut === "") return null; // Champ vide  
  
    // parseFloat gère virgule ou point selon locale ?  
    // ATTENTION : JS standard n'accepte QUE le point '.' comme séparateur décimal.  
    // Remplacer la virgule par un point si utilisateur FR :  
    let normalise = brut.replace(/,/g, '.');  
  
    let nombre = parseFloat(normalise);  
  
    if (isNaN(nombre)) {  
        alert("Veuillez entrer un nombre valide (ex : 12.5 ou 12,5).");  
        champ.focus();  
        return null;  
    }  
    return nombre;  
}
```

6.4 Théorie des conversions linéaires : fonctions affines

PROPRIÉTÉ Modèle mathématique des conversions d'unités

Toute conversion entre deux échelles proportionnelles (devises, longueurs, masses) ou affine (températures °C/°F) s'exprime par une fonction affine :

$$f(x) = a \cdot x + b$$

- Conversion proportionnelle (devise, m ↔ km) : $b = 0$. a = taux.
- Conversion affine (température) : $a = \frac{9}{5}$, $b = 32$ pour °C → °F.

La programmation consiste à identifier les coefficients a et b pour chaque paire d'unités.

6.5 Convertisseur de devises : FCFA ↔ EUR

Le taux de change officiel (zone CEMAC) est fixe : 1 EUR = 655,957 FCFA (arrondi souvent à 655,96 pour l'affichage).

MÉTHODE Créer un convertisseur de devises générique

1. Définir la constante `TAUX` (FCFA pour 1 EUR).
2. Créer une fonction `convertirDevise(montant, sens)` où `sens` vaut `"FCFA_VERS_EUR"` ou `"EUR_VERS_FCFA"`.
3. Appliquer la formule :
 - FCFA → EUR : $\text{montant} / \text{TAUX}$
 - EUR → FCFA : $\text{montant} \times \text{TAUX}$
4. Formater le résultat (2 décimales pour EUR, 0 ou 3 pour FCFA) avec `toFixed(n)`.

EXEMPLE Code complet : Convertisseur FCFA/EUR

```
const TAUX_EUR_FCFA = 655.957; // 1 EUR = 655.957 FCFA

/**
 * Convertit un montant entre FCFA et EUR.
 * @param {number} montant - Valeur numérique positive.
 * @param {string} sens - "FCFA_EUR" ou "EUR_FCFA".
 * @returns {string} - Résultat formaté avec unité.
 */
function convertirDevise(montant, sens) {
  let resultat;
  let unite;

  if (sens === "FCFA_EUR") {
    resultat = montant / TAUX_EUR_FCFA;
    unite = " EUR";
    // Arrondi à 2 décimales (centimes d'euro)
    return resultat.toFixed(2) + unite;
  }
  else if (sens === "EUR_FCFA") {
    resultat = montant * TAUX_EUR_FCFA;
    unite = " FCFA";
  }
}
```

```
// Arrondi à l'entier (pas de centime FCFA) ou 3 décimales
return Math.round(resultat).toLocaleString('fr-FR') + unite;
}
return "Sens invalide";
}

// Test console
console.log(convertirDevise(10000, "FCFA_EUR")); // "15.24 EUR"
console.log(convertirDevise(15.24, "EUR_FCFA")); // "10 000 FCFA" (approx)
```

6.6 Convertisseur de températures : °C ↔ °F ↔ K

Formules exactes :

$$F = C \times \frac{9}{5} + 32 \quad (\text{°C vers °F})$$

$$C = (F - 32) \times \frac{5}{9} \quad (\text{°F vers °C})$$

$$K = C + 273.15 \quad (\text{°C vers Kelvin})$$

$$C = K - 273.15 \quad (\text{Kelvin vers °C})$$

MÉTHODE Stratégie « Pivot Celsius » pour N unités

Plutôt que d'écrire $N \times (N - 1)$ fonctions, on convertit tout vers une unité pivot (ici °C), puis du pivot vers la cible. Cela ne demande que $2N$ fonctions.

1. `versCelsius(valeur, uniteSource)` : renvoie °C.
2. `depuisCelsius(valeurCelsius, uniteCible)` : renvoie valeur dans cible.
3. `convertirTemp(valeur, source, cible)` : orchestration.

EXEMPLE Convertisseur température générique

```
function versCelsius(valeur, unite) {
  switch (unite) {
    case "C": return valeur;
    case "F": return (valeur - 32) * 5/9;
    case "K": return valeur - 273.15;
    default: return NaN;
  }
}

function depuisCelsius(celsius, unite) {
```

```
switch (unite) {
  case "C" : return celsius;
  case "F" : return celsius * 9/5 + 32;
  case "K" : return celsius + 273.15;
  default : return NaN;
}

function convertirTemperature(valeur, source, cible) {
  if (source === cible) return valeur.toFixed(2) + " " + cible;

  let c = versCelsius(valeur, source);
  if (isNaN(c)) return "Unité source inconnue";

  let res = depuisCelsius(c, cible);
  if (isNaN(res)) return "Unité cible inconnue";

  // Affichage : 2 décimales pour C/F, 3 pour K (précision scientifique)
  let precision = (cible === "K") ? 3 : 2;
  return res.toFixed(precision) + " " + cible;
}

// Tests
console.log(convertirTemperature(100, "C", "F")); // "212.00 F"
console.log(convertirTemperature(32, "F", "C")); // "0.00 C"
console.log(convertirTemperature(0, "C", "K")); // "273.150 K"
```

6.7 Convertisseur de dates et objet Date

L'objet `Date` est l'outil natif pour manipuler dates et heures. Attention : les mois sont indexés à 0 (Janvier = 0, Décembre = 11).

PROPRIÉTÉ Méthodes principales de `Date` (locale)

Méthode	Retour	Note
getFullYear()	number	Année sur 4 chiffres (ex : 2025)
getMonth()	number	0–11
getDate()	number	Jour du mois (1–31)
getDay()	number	Jour de la semaine (0=Dim...6=Sam)
getHours(), getMinutes(), getSeconds()	number	Heure locale
toLocaleDateString(locale, options)	string	Formatage localisé puissant
toISOString()	string	Format ISO 8601 UTC (ex : "2025-01-15T14:30:00.000Z")

LESAVAIS-TU ? L'API Intl : Formatage professionnel

L'objet `Intl.NumberFormat` et `Intl.DateTimeFormat` permettent d'afficher nombres et dates selon les conventions locales (séparateurs, devise, fuseau) sans bibliothèque externe.

```
let montant = 1234567.89;
// Affichage FCFA (XAF) selon norme française (espace insécable comme séparateur milliers)
console.log(new Intl.NumberFormat('fr-FR', {
  style: 'currency',
  currency: 'XAF',
  minimumFractionDigits: 0
}).format(montant));
// "1 234 568 FCFA" (arrondi, espace insécable)

let maintenant = new Date();
console.log(new Intl.DateTimeFormat('fr-FR', {
  weekday: 'long', year: 'numeric', month: 'long', day: 'numeric'
}).format(maintenant));
// "mercredi 15 janvier 2025"
```

EXEMPLE Convertisseur de format de date

```
function formaterDateFrancaise(dateObj) {
  // Options pour format long : "mercredi 15 janvier 2025"
  const options = {
    weekday: 'long',
    year: 'numeric',
    month: 'long',
```

```

        day : 'numeric'
    };
    return dateObj.toLocaleDateString('fr-FR', options);
}

function formaterDateISO(dateObj) {
    // Format ISO court : "2025-01-15"
    return dateObj.toISOString().split('T')[0];
}

// Exemple : Saisie via <input type="date"> (renvoie ISO "YYYY-MM-DD")
function afficherDetailsDate(isoString) {
    let d = new Date(isoString + "T00:00:00"); // Évite décalage fuseau
    if (isNaN(d.getTime())) return "Date invalide";

    return {
        francais : formaterDateFrancaise(d),
        iso : formaterDateISO(d),
        jourSemaine : d.toLocaleDateString('fr-FR', {weekday : 'long'}),
        timestamp : d.getTime() // Millisecondes depuis 1970
    };
}

```

6.8 Intégration complète : Application « Convertisseur Universel »

Nous assemblons maintenant toutes les briques : formulaire HTML sémantique, saisie `<input type="number">`, sélection `<select>`, bouton déclencheur `onclick`, et zone de sortie `<output>`.

EXPÉRIENCE TP Guidé : Développement du Convertisseur Universel

Objectif : Créer une page unique permettant de convertir devises, températures ou dates. **Outils :** Éditeur de code (VS Code, Notepad++), Navigateur (Chrome/Firefox). **Durée :** 45 min.

Étape 1 : Structure HTML (index.html) Créer le squelette avec trois sections masquées/affichées selon le type choisi, ou un formulaire unique adaptatif. Ici, version unique adaptative :

```

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Convertisseur Universel – Lycée</title>
    <style>
        body { font-family : Arial, sans-serif; max-width : 600px; margin : 2rem auto; padding : 1rem; }
        .groupe { margin-bottom : 1rem; display : flex; gap : 0.5rem; align-items : center; flex-wrap : wrap; }
    </style>

```

```

    label { min-width : 120px; font-weight : bold; }
    input, select { padding : 0.5rem; flex : 1; min-width : 150px; }
    button { background : #0066cc; color : white; border : none; padding : 0.7rem 1.5rem; cursor :
    pointer; font-size : 1rem; }
    button :hover { background : #004499; }
    output { display : block; margin-top : 1.5rem; padding : 1rem; background : #f0f8ff; border : 1px
    solid #0066cc; font-size : 1.2rem; font-weight : bold; text-align : center; min-height : 2em; }
    .cache { display : none; }
</style>
</head>
<body>
  <h1>Convertisseur Universel</h1>

  <!-- Choix de la catégorie -->
  <div class="groupe">
    <label for="typeConversion">Type :</label>
    <select id="typeConversion" onchange="adapterFormulaire()">
      <option value="devise">Devises (FCFA <-> EUR)</option>
      <option value="temperature">Températures (°C, °F, K)</option>
      <option value="date">Formatage Date</option>
    </select>
  </div>

  <!-- Zone de saisie dynamique (gérée en JS) -->
  <div id="zoneSaisie" class="groupe">
    <!-- Injecté par JS -->
  </div>

  <button onclick="lancerConversion()">Convertir</button>

  <output id="resultat" for="valeur uniteSource uniteCible">—</output>

  <script src="convertisseur.js"></script>
</body>
</html>

```

Étape 2 : Logique JavaScript (convertisseur.js)

```

// --- CONSTANTES & CONFIG ---
const TAUX_EUR_FCFA = 655.957;
const UNITES = {
  devise : { source : ["FCFA", "EUR"], cible : ["EUR", "FCFA"] },
  temperature : { source : ["C", "F", "K"], cible : ["C", "F", "K"] },
  date : { source : ["ISO", "FR_LONG"], cible : ["FR_LONG", "ISO", "TIMESTAMP"] }
};

// --- FONCTIONS UTILITAIRES ---
function peuplerSelect(idSelect, options, valeurDefaut) {
  let sel = document.getElementById(idSelect);
  sel.innerHTML = ""; // Vider

```

```
options.forEach(opt => {
    let o = document.createElement("option");
    o.value = opt; o.textContent = opt;
    if (opt === valeurDefaut) o.selected = true;
    sel.appendChild(o);
});
}

function lireNombreSecurise(id) {
    let val = document.getElementById(id).value.trim().replace(/,/g, '.');
    let n = parseFloat(val);
    return isNaN(n) ? null : n;
}

// --- LOGIQUE MÉTIER (Pures) ---
function convertirDevise(montant, source, cible) {
    if (source === cible) return montant.toFixed(2) + " " + cible;
    let eur = (source === "FCFA") ? montant / TAUX_EUR_FCFA : montant;
    let res = (cible === "FCFA") ? eur * TAUX_EUR_FCFA : eur;
    return (cible === "FCFA" ? Math.round(res) : res.toFixed(2)) + " " + cible;
}

function convertirTemp(val, src, cib) {
    if (src === cib) return val.toFixed(2) + " " + cib;
    // Vers Celsius
    let c = (src === "C") ? val : (src === "F") ? (val - 32) * 5 / 9 : val - 273.15;
    // Depuis Celsius
    let res = (cib === "C") ? c : (cib === "F") ? c * 9 / 5 + 32 : c + 273.15;
    return res.toFixed(cib === "K" ? 3 : 2) + " " + cib;
}

function convertirDate(isoStr, cible) {
    let d = new Date(isoStr + "T00:00:00"); // Minuit local pour éviter décalage UTC
    if (isNaN(d)) return "Date invalide";
    switch(cible) {
        case "FR_LONG" : return d.toLocaleDateString('fr-FR', {weekday : 'long', year : 'numeric', month : 'long', day : 'numeric'});
        case "ISO" : return d.toISOString().split('T')[0];
        case "TIMESTAMP" : return d.getTime();
        default : return "Format inconnu";
    }
}

// --- CONTRÔLEUR PRINCIPAL ---
function adapterFormulaire() {
    let type = document.getElementById("typeConversion").value;
    let zone = document.getElementById("zoneSaisie");
    let html = "";

    if (type === "devise" || type === "temperature") {
```

```
let u = UNITES[type];
html = `
  <label for="valeur">Valeur :</label>
  <input type="text" id="valeur" placeholder="Ex : 10000 ou 25,5" required>
  <label for="uniteSource">De :</label>
  <select id="uniteSource"></select>
  <label for="uniteCible">Vers :</label>
  <select id="uniteCible"></select>
`;
zone.innerHTML = html;
peuplerSelect("uniteSource", u.source, u.source[0]);
peuplerSelect("uniteCible", u.cible, u.cible[0]);
}
else if (type === "date") {
  html = `
    <label for="valeur">Date (AAAA-MM-JJ) :</label>
    <input type="date" id="valeur" required>
    <label for="uniteCible">Format cible :</label>
    <select id="uniteCible"></select>
  `;
  zone.innerHTML = html;
  peuplerSelect("uniteCible", UNITES.date.cible, "FR_LONG");
  // Pré-remplir avec aujourd'hui
  document.getElementById("valeur").value = new Date().toISOString().split('T')[0];
}
}

function lancerConversion() {
  let type = document.getElementById("typeConversion").value;
  let sortie = document.getElementById("resultat");
  let texte = "Erreur de saisie.";

  try {
    if (type === "devise" || type === "temperature") {
      let val = lireNombreSecurise("valeur");
      if (val === null) throw "Nombre invalide";
      let src = document.getElementById("uniteSource").value;
      let cib = document.getElementById("uniteCible").value;

      if (type === "devise") texte = convertirDevise(val, src, cib);
      else texte = convertirTemp(val, src, cib);
    }
    else if (type === "date") {
      let val = document.getElementById("valeur").value; // Format ISO natif
      if (!val) throw "Date manquante";
      let cib = document.getElementById("uniteCible").value;
      texte = convertirDate(val, cib);
    }
    sortie.style.color = "green";
  } catch (e) {
```

```

    texte = "Erreur : " + e;
    sortie.style.color = "red";
  }
  sortie.textContent = texte;
}

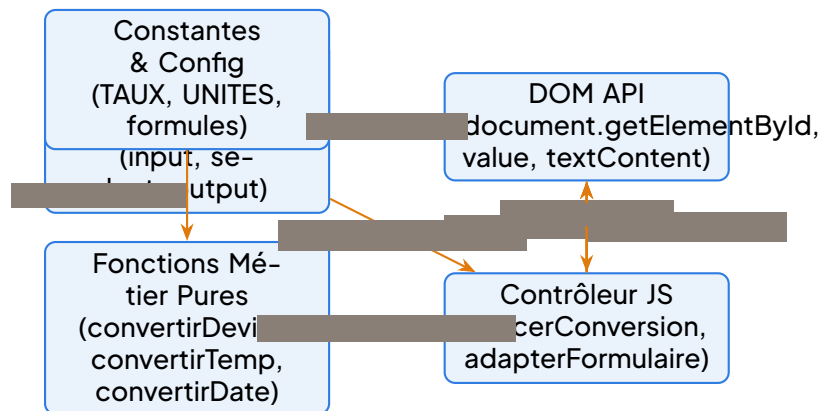
// Initialisation au chargement
document.addEventListener("DOMContentLoaded", adapterFormulaire);

```

Étape 3 : Tests et observations

1. Ouvrir `index.html` dans le navigateur.
2. Tester chaque onglet : Devise (10000 FCFA → EUR), Température (100 C → F), Date (aujourd'hui → FR_LONG).
3. Vérifier la gestion d'erreurs : saisir "abc" dans valeur numérique, laisser champ vide.
4. Observer la mise à jour dynamique des listes `<select>` via `adapterFormulaire()`.

Interprétation : La séparation HTML/CSS/JS, l'utilisation de `textContent` pour la sécurité, la factorisation du code métier (fonctions pures) et le contrôleur unique `lancerConversion` illustrent une architecture propre, maintenable et robuste.



Flux de données : Entrée → Contrôle → Traitement → Sortie

Architecture du Convertisseur Universel : séparation des responsabilités (Vue/Contrôleur/Modèle) et flux de données unidirectionnel.

6.9 Exercice résolu : Convertisseur d'unités de longueur

EXERCICE RÉSOLU 6

Convertisseur Mètres ↔ Pieds ↔ Pouces

Énoncé : Écrire une fonction JavaScript `convertirLongueur(valeur, uniteSource,`

`uniteCible`) gérant les unités : "m" (mètre), "ft" (pied), "in" (pouce). Rappels : 1 m = 3.28084 ft ; 1 ft = 12 in. Afficher le résultat avec 4 décimales pour les mètres, 2 pour les pieds/pouces.

SOLUTION

Solution détaillée :

1. Analyse mathématique (Pivot Mètre) :

$$\begin{aligned} \text{Vers mètre : } m &= \begin{cases} val & \text{si } src = "m" \\ val / 3.28084 & \text{si } src = "ft" \\ val / (3.28084 \times 12) & \text{si } src = "in" \end{cases} \\ \text{Depuis mètre : } res &= \begin{cases} m & \text{si } cib = "m" \\ m \times 3.28084 & \text{si } cib = "ft" \\ m \times 3.28084 \times 12 & \text{si } cib = "in" \end{cases} \end{aligned}$$

2. Implémentation JavaScript :

```
const PIED_PAR_METRE = 3.28084;
const POUCES_PAR_PIED = 12;

function versMetre(val, unite) {
  switch (unite) {
    case "m": return val;
    case "ft": return val / PIED_PAR_METRE;
    case "in": return val / (PIED_PAR_METRE * POUCES_PAR_PIED);
    default: return NaN;
  }
}

function depuisMetre(m, unite) {
  switch (unite) {
    case "m": return m;
    case "ft": return m * PIED_PAR_METRE;
    case "in": return m * PIED_PAR_METRE * POUCES_PAR_PIED;
    default: return NaN;
  }
}

function convertirLongueur(valeur, source, cible) {
  if (source === cible) return formater(valeur, cible);

  let metres = versMetre(valeur, source);
  if (isNaN(metres)) return "Unité source invalide";

  let resultat = depuisMetre(metres, cible);
  if (isNaN(resultat)) return "Unité cible invalide";

  return formater(resultat, cible);
}
```

```

}

function formater(nombre, unite) {
  // Précision adaptée à l'unité
  let decimales = (unite === "m") ? 4 : 2;
  return nombre.toFixed(decimales) + " " + unite;
}

// Tests
console.log(convertirLongueur(1, "m", "ft")); // "3.28 ft"
console.log(convertirLongueur(12, "in", "m")); // "0.3048 m"
console.log(convertirLongueur(3.28084, "ft", "in")); // "39.37 in"

```

3. Points clés validés :

- Utilisation de constantes nommées (PIED_PAR_METRE) pour la lisibilité.
- Stratégie Pivot (Mètre) évitant les 6 conversions croisées.
- Fonction `formater` externalisée pour la présentation.
- Gestion des unités identiques (retour direct) et invalides (NaN).

EXERCICE 5 Entraînement : Convertisseur d'aires

Écrire une page HTML/JS complète (fichier unique) permettant de convertir des surfaces entre m^2 , ha (hectare : 10 000 m^2), are (100 m^2) et km^2 (1 000 000 m^2).

1. Interface : `<input type="number" step="any">`, deux `<select>`, `<button>`, `<output>`.
2. JS : Fonction `convertirAire(val, src, cib)` utilisant le pivot m^2 .
3. Affichage : Notation scientifique (ex : 1.5e-4 km^2) ou `toLocaleString` pour grands nombres. Gérer NaN.

CORRIGÉ Exercice : Convertisseur d'aires

```

<!DOCTYPE html><html lang="fr"><head><meta charset="UTF-8"><title>Aires</title>
<style>body{max-width:500px;margin:2rem auto;font-family:sans-serif}.g{display:flex;gap:.5rem;
margin:.5rem 0;align-items:center}label{min-width:80px}input,select{flex:1;padding:.4rem}
button{width:100%;padding:.6rem;background:#28a745;color:#fff;border:none;cursor:pointer}
output{display:block;margin-top:1rem;padding:1rem;background:#e8f5e9;border:1px solid
#28a745;font-weight:bold;text-align:center}</style>
</head><body>
<h2>Convertisseur d'Aires (Pivot m²)</h2>
<div class="g"><label>Valeur</label><input type="number" step="any" id="val" placeholder="Ex :
10000"></div>

```

```
<div class="g"><label>De</label><select id="src"><option>m²</option><option>are</option><option>ha</option><option>km²</option></select></div>
<div class="g"><label>Vers</label><select id="cib"><option>ha</option><option>m²</option><option>are</option><option>km²</option></select></div>
<button onclick="convertir()">Convertir</button>
<output id="res">—</output>

<script>
const VERS_M2 = { "m²" : 1, "are" : 100, "ha" : 10000, "km²" : 1000000 };

function convertir() {
  let v = parseFloat(document.getElementById("val").value);
  let s = document.getElementById("src").value;
  let c = document.getElementById("cib").value;
  let out = document.getElementById("res");

  if (isNaN(v)) { out.textContent = "Entrez un nombre"; out.style.color = "red"; return; }
  if (s === c) { out.textContent = v + " " + c; out.style.color = "green"; return; }

  let m2 = v * VERS_M2[s]; // Vers pivot
  let res = m2 / VERS_M2[c]; // Depuis pivot

  // Affichage intelligent
  let txt = (res >= 10000 || (res < 0.001 && res !== 0)) ? res.toExponential(4) : res.toLocaleString('fr-FR', {maximumFractionDigits: 6});
  out.textContent = txt + " " + c;
  out.style.color = "green";
}
</script>
</body></html>
```

À RETENIR Synthèse : Entrées/Sorties & Convertisseurs

1. Entrées : `prompt()` (dépannage), `input.value` (standard), `confirm()` (booléen). Toujours convertir (`parseFloat`, `Number`) avant calcul.
2. Sorties : `alert()` (dépannage), `element.textContent` (sécurisé, texte), `element.innerHTML` (HTML, attention XSS), `<output>` (sémantique formulaires).
3. Convertisseurs : Modéliser par fonction affine $f(x) = ax + b$. Stratégie Pivot (unité de base) pour factoriser le code.
4. Devises : Taux fixe FCFA/EUR = 655,957. `toFixed(2)` pour EUR, `Math.round + toLocaleString` pour FCFA.
5. Températures : Pivot °C. Formules exactes $F = C \times 9/5 + 32$, $K = C + 273.15$.
6. Dates : Objet `Date`. Mois 0–11. `toLocaleDateString` pour affichage localisé. `Intl` pour

formatage pro.

7. Architecture : Séparer HTML (Vue), JS Contrôleur (Événements, DOM), JS Métier (Fonctions pures, testables).

7 ACTIVITÉ D'INTÉGRATION

7.1 Objectifs de l'activité

À l'issue de cet atelier en binôme, tu seras capable de :

- mobiliser l'**ensemble des savoir-faire de la leçon** dans un projet unique et fonctionnel : déclaration de variables, conversions de type avec `parseInt()` et `parseFloat()`, opérateurs arithmétiques, de comparaison et logiques, structures conditionnelles et itératives, déclaration et appel de fonctions ;
- structurer une page Web en séparant le contenu (HTML) du comportement (JavaScript placé dans un fichier externe lié par `<script src="..." defer>`);
- gérer les **entrées/sorties** : lecture des champs de saisie via `value`, affichage du résultat via `textContent`, messages d'erreur avec `alert()` et confirmation avec `confirm()` ;
- valider les données saisies (test `isNaN()`, vérification des unités) avant tout calcul, afin de produire un programme **robuste** ;
- adopter une démarche de **décomposition fonctionnelle** : une fonction principale qui aiguille le travail vers des fonctions spécialisées.

7.2 Situation

La coopérative des commerçants du marché central de Bafoussam vient d'ouvrir un petit **cybercafé-comptoir** où les clients peuvent imprimer des documents et consulter Internet. Son gérant, M. Kamdem, constate que ses clients posent sans cesse les mêmes questions :

- « 50 000 FCFA, ça fait combien en euros ? Et en dollars ? » (taux pratiqués au comptoir : 1 € = 656 FCFA ; 1 \$ = 600 FCFA) ;
- « Le bulletin météo annonce 86 degrés Fahrenheit à Douala, c'est chaud ou pas ? » ;
- « Mon bon de commande est valable 90 jours à partir du 15 mars 2025, il expire quand ? ».

M. Kamdem demande à la classe de Première C du lycée voisin de lui concevoir une page Web unique, nommée `convertisseur.html`, utilisable **hors connexion** sur le poste du comptoir. La page devra proposer :

1. un **menu déroulant** pour choisir le type de conversion : Devises, Températures ou Dates ;
2. une **zone de saisie** pour la valeur à convertir, une liste pour l'**unité source** et une liste pour l'**unité cible** ;
3. un bouton « **Convertir** » dont l'événement `onclick` déclenche le calcul ;
4. l'affichage du résultat dans une balise `<output>`, sans recharger la page ;
5. des messages clairs en cas d'erreur de saisie, et une demande de confirmation avant chaque conversion.

Pour les devises, les conversions possibles sont : FCFA → Euro, FCFA → Dollar, Euro → FCFA, Dollar → FCFA. Pour les températures : Celsius → Fahrenheit et Fahrenheit → Celsius, avec les formules $F = \frac{9}{5}C + 32$ et $C = \frac{5}{9}(F - 32)$. Pour les dates : le client saisit une date de départ et un nombre de jours n ; la page calcule la date d'arrivée en ajoutant n jours (on utilisera l'objet `Date` de JavaScript).

7.3 Consignes

Travail en binôme. Durée : 2 h. Livrables : deux fichiers, `convertisseur.html` et `convertisseur.js`, placés dans le même dossier.

1. **Analyse.** Identifie, pour chacun des trois types de conversion, les données d'entrée, les données de sortie et les traitements. Rédige le dictionnaire des données du programme.
2. **Maquette HTML.** Crée la page `convertisseur.html` : titre, menu déroulant `<select id="type">`, champ numérique `<input id="valeur">`, deux listes `<select id="source">` et `<select id="cible">`, bouton `<button onclick="convertir()">` et balise `<output id="resultat">`. Lie le script externe avec l'attribut `defer`.
3. **Fonctions spécialisées.** Dans `convertisseur.js`, déclare trois fonctions : `convertirDevises(montant, source, cible)`, `convertirTemperature(valeur, source, cible)` et `convertirDate(dateDepart, nbJours)`. Chacune retourne une chaîne de caractères contenant le résultat.
4. **Fonction principale.** Écris la fonction `convertir()` qui : récupère les valeurs des champs ; demande confirmation avec `confirm()` ; valide la saisie avec `isNaN()` (et signale toute erreur par `alert()`) ; aiguille vers la bonne fonction spécialisée grâce à une structure conditionnelle ; affiche le résultat via `textContent`.
5. **Conversions de type.** Justifie, dans un commentaire du code, l'emploi de `parseFloat()` pour le montant et de `parseInt()` pour le nombre de jours.
6. **Tests.** Teste les cas suivants et consigne les résultats dans un tableau : 50 000 FCFA en euros ; 100 € en FCFA ; 86 °F en Celsius ; 37 °C en Fahrenheit ; 15/03/2025 +90 jours ; saisie « abc » comme montant ; champ valeur laissé vide.
7. **Ergonomie (bonus).** Propose une amélioration : par exemple, adapter automatiquement les listes d'unités au type choisi grâce à l'événement `onchange` du menu.

7.4 Ressources à mobiliser

À RETENIR

Boîte à outils de la leçon

- **Entrées :** `prompt()` (chaîne), `document.getElementById("id").value` (chaîne) ; **conversions :** `parseFloat()` (réel), `parseInt()` (entier) ; **validation :** `isNaN(x)` vaut `true` si `x` n'est pas un nombre.
- **Sorties :** `alert(msg)`, `confirm(msg)` (retourne `true/false`), `document.getElementById("id").textContent = msg`.

- **Opérateurs** : + - * /, comparaison === !== < >, logiques && ||, affectation = += -= *= /=, incrémentation ++ --.
- **Structures** : if / else if / else, switch, boucles for et while.
- **Fonctions** : `function nom(p1, p2) { ... return r; }`; déclenchement par l'événement `onclick`.
- **Formules** : $F = \frac{9}{5}C + 32$; $C = \frac{5}{9}(F - 32)$; taux : 1€ = 656 FCFA, 1\$ = 600 FCFA.

7.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Multi-Convertisseur Web — corrigé complet

Construire la page demandée par M. Kamdem, en respectant les consignes 1 à 6.

SOLUTION

Étape 1 — Dictionnaire des données. On distingue les variables globales du problème :

Nom	Type	Rôle
type	chaîne	type de conversion choisi (devises, temperature, date)
valeur	réel	valeur numérique saisie (montant, température)
source, cible	chaînes	unités de départ et d'arrivée
resultat	chaîne	message affiché dans la balise <output>

Étape 2 — La page HTML. Le fichier `convertisseur.html` ne contient que la structure ; le comportement est déporté dans le fichier `.js`, lié avec `defer` pour garantir que la page est chargée avant l'exécution du script.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Multi-Convertisseur – Cybercafe de Bafoussam</title>
  <script src="convertisseur.js" defer></script>
</head>
<body>
  <h1>Multi-Convertisseur du comptoir</h1>

  <label for="type">Type de conversion :</label>
  <select id="type">
    <option value="devises">Devises</option>
    <option value="temperature">Temperatures</option>
    <option value="date">Dates</option>
```

```

</select>
<br><br>

<label for="valeur">Valeur :</label>
<input type="text" id="valeur" placeholder="Ex : 50000">

<select id="source">
  <option value="FCFA">FCFA</option>
  <option value="EUR">Euro</option>
  <option value="USD">Dollar</option>
  <option value="C">Celsius</option>
  <option value="F">Fahrenheit</option>
</select>

<select id="cible">
  <option value="FCFA">FCFA</option>
  <option value="EUR">Euro</option>
  <option value="USD">Dollar</option>
  <option value="C">Celsius</option>
  <option value="F">Fahrenheit</option>
</select>

<label for="jours">Jours (dates) :</label>
<input type="text" id="jours" placeholder="Ex : 90">
<br><br>

<button onclick="convertir()">Convertir</button>
<br><br>
<strong>Resultat : </strong><output id="resultat">---</output>
</body>
</html>

```

Étape 3 — Les fonctions spécialisées. Chaque fonction reçoit des paramètres déjà validés et retourne une chaîne : elle ne s'occupe ni de la saisie ni de l'affichage (principe de décomposition).

```

// ===== Fonctions specialisees =====

function convertirDevises(montant, source, cible) {
  var resultat;
  if (source === "FCFA" && cible === "EUR") {
    resultat = montant / 656;
    return montant + " FCFA = " + resultat.toFixed(2) + " EUR";
  } else if (source === "FCFA" && cible === "USD") {
    resultat = montant / 600;
    return montant + " FCFA = " + resultat.toFixed(2) + " USD";
  } else if (source === "EUR" && cible === "FCFA") {
    resultat = montant * 656;
    return montant + " EUR = " + resultat + " FCFA";
  } else if (source === "USD" && cible === "FCFA") {

```

```
    resultat = montant * 600;
    return montant + " USD = " + resultat + " FCFA";
  } else {
    return "Conversion de devises non prevue.";
  }
}

function convertirTemperature(valeur, source, cible) {
  var resultat;
  if (source === "C" && cible === "F") {
    resultat = (9 / 5) * valeur + 32;
    return valeur + " C = " + resultat.toFixed(1) + " F";
  } else if (source === "F" && cible === "C") {
    resultat = (5 / 9) * (valeur - 32);
    return valeur + " F = " + resultat.toFixed(1) + " C";
  } else {
    return "Conversion de temperature non prevue.";
  }
}

function convertirDate(dateDepart, nbJours) {
  var d = new Date(dateDepart); // format AAAA-MM-JJ
  if (isNaN(d.getTime())) {
    return "Date de depart invalide.";
  }
  d.setDate(d.getDate() + nbJours); // ajout des jours
  return "Date d'arrivee : " + d.toLocaleDateString("fr-FR");
}
```

Étape 4 — La fonction principale. Elle orchestre tout : lecture, confirmation, validation, aiguillage, affichage.

```
// ===== Fonction principale (evenement onclick) =====

function convertir() {
  // 1. Recuperation des valeurs (toujours des chaines)
  var type = document.getElementById("type").value;
  var source = document.getElementById("source").value;
  var cible = document.getElementById("cible").value;
  var message;

  // 2. Confirmation avant traitement
  if (!confirm("Lancer la conversion ?")) {
    return; // l'utilisateur a annule
  }

  // 3. Aiguillage selon le type choisi
  if (type === "devises" || type === "temperature") {
    var valeur = parseFloat(document.getElementById("valeur").value);
    // parseFloat : montants et temperatures peuvent etre decimaux
    if (isNaN(valeur)) {
```

```

    alert("Erreur : saisissez une valeur numerique !");
    return;
}
if (type === "devises"){
    message = convertirDevises(valeur, source, cible);
} else {
    message = convertirTemperature(valeur, source, cible);
}
} else { // type === "date"
    var dateDepart = document.getElementById("valeur").value;
    var nbJours = parseInt(document.getElementById("jours").value, 10);
    // parseInt : un nombre de jours est forcément entier
    if (isNaN(nbJours) || nbJours < 0){
        alert("Erreur : nombre de jours invalide !");
        return;
    }
    message = convertirDate(dateDepart, nbJours);
}

// 4. Affichage du resultat sans recharger la page
document.getElementById("resultat").textContent = message;
}

```

Étape 5 — Table des tests.

Saisie	Résultat attendu
50 000 FCFA → EUR	50000 FCFA = 76,22 EUR
100 EUR → FCFA	100 EUR = 65600 FCFA
86 F → C	86 F = 30,0 C
37 C → F	37 C = 98,6 F
2025-03-15 +90 jours	Date d'arrivée : 13/06/2025
« abc » comme montant	alert : valeur numérique exigée

Vérification manuelle : $50\,000 / 656 \approx 76,22$; $\frac{5}{9}(86-32) = \frac{5}{9} \times 54 = 30$; $\frac{9}{5} \times 37 + 32 = 66,6 + 32 = 98,6$. Le programme est conforme.

ATTENTION Pièges fréquents rencontrés pendant l'atelier

- Oublier `parseFloat()` : "50000" / 656 fonctionne parfois par conversion implicite, mais "100" + "200" donne "100200" (concaténation) et non 300. Convertis toujours explicitement.
- Utiliser `=` (affectation) au lieu de `===` (comparaison) dans les tests : `if (source = "FCFA")` est toujours vrai et fausse tout le programme.

- Oublier l'attribut `defer` ou placer le script avant les éléments HTML : `getElementById` retourne alors `null` et la page plante silencieusement.
- Tester le nombre avant conversion : `isNaN("abc")` vaut `true`, mais c'est la valeur convertie qu'il faut tester après `parseFloat()`.

7.6 Bilan de l'activité

Cet atelier a permis de mettre en évidence plusieurs acquis essentiels de la leçon :

- la **complémentarité HTML/JavaScript** : le HTML, statique, décrit la page ; seul le JavaScript la rend interactive grâce aux événements comme `onclick` ;
- l'intérêt de la **décomposition en fonctions** : la fonction principale `convertir()` reste courte et lisible parce que chaque calcul est confié à une fonction spécialisée réutilisable et testable isolément ;
- la nécessité de la **validation des entrées** : un programme destiné à un vrai utilisateur (ici, les clients de M. Kamdem) doit prévoir les saisies incorrectes grâce à `isNaN()`, `alert()` et `confirm()` ;
- l'importance des **conversions de type** : toute donnée venant d'un champ de formulaire est une chaîne, et `parseFloat()/parseInt()` sont indispensables avant tout calcul ;
- la séparation des fichiers (`.html` / `.js`) comme bonne pratique professionnelle, qui facilite la maintenance : M. Kamdem pourra faire modifier les taux de change dans le seul fichier JavaScript sans toucher à la page.

Le multi-convertisseur obtenu est un premier vrai logiciel Web : il pourra être enrichi en classe de Terminale (nouvelles devises, interface améliorée, historique des conversions), illustrant ainsi la démarche de développement par étapes.

8 MÉTHODES ET EXERCICES RÉSOLUS

8.1 Fiches méthodes et exercices résolus

MÉTHODE Insérer du JavaScript dans une page HTML

Pour rendre une page HTML interactive avec JavaScript, procède ainsi :

1. Ouvre la balise `<script>` et ferme-la avec `</script>` : tout le code JavaScript se place entre les deux. La balise peut figurer dans `<head>` (chargement avant l'affichage) ou dans `<body>` (exécution au fil de la lecture).
2. Écris les instructions JavaScript, chacune terminée par un **point-virgule** ; (bonne pratique exigée à l'examen, bien que JS tolère parfois l'omission).
3. Pour afficher un résultat dans le flux de la page, utilise `document.write("texte")` ; pour une boîte de dialogue modale, `alert("texte")` ; pour une boîte de confirmation, `confirm("question")` (renvoie `true` ou `false`) ; pour une saisie, `prompt("message")` (renvoie une chaîne ou `null`).
4. Enregistre le fichier avec l'extension `.html` et ouvre-le dans un navigateur pour tester.
5. Pour isoler le script, on peut aussi le placer dans un fichier `.js` externe appelé par `<script src="fichier.js"></script>` (à placer de préférence avant `</body>`).

Réflexe Bac : si l'énoncé demande de « compléter une page », repère où placer `<script>` et vérifie que chaque balise ouverte est fermée.

EXERCICE RÉSOLU 8 Première page dynamique

Énoncé. Le proviseur d'un lycée de Yaoundé veut afficher sur la page d'accueil du site de l'établissement le message « Bienvenue au Lycée de Nkol-Eton ! » dans une boîte de dialogue, puis « Année scolaire 2024–2025 » dans le corps de la page. Écris le code HTML/JavaScript complet de cette page.

SOLUTION

Solution détaillée.

1. On structure d'abord le squelette HTML minimal : `<html>`, `<head>` (avec le titre et le charset) et `<body>`.
2. Le message en boîte de dialogue est une sortie : on utilise `alert()`. Le texte dans la page est aussi une sortie : on utilise `document.write()`.
3. On place les deux instructions dans une balise `<script>` située dans le corps, chaque instruction terminée par ; .

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Accueil du lycee</title>
  </head>
  <body>
    <script>
      alert("Bienvenue au Lycee de Nkol-Eton !");
      document.write("Annee scolaire 2024-2025");
    </script>
  </body>
</html>
```

Vérification mentale : à l'ouverture du fichier, le navigateur exécute le script ligne par ligne : la boîte `alert` apparaît d'abord (le script est bloqué tant qu'on ne clique pas sur OK), puis le texte s'écrit dans la page. Conclusion : la page affiche successivement la boîte de bienvenue puis le texte de l'année scolaire : le HTML seul était incapable de produire la boîte de dialogue, d'où l'apport de JavaScript.

MÉTHODE**Déclarer des variables et convertir les types**

1. **Déclare** chaque variable avec le mot-clé `var` suivi du nom (sans espace, sans accent, commençant par une lettre ou `_` ou `$`) : `var age`; (notation `var` imposée par le programme de 1ère).
2. **Affecte** une valeur avec `=` : `age = 17`; ou directement `var age = 17`;
3. Retiens les trois types de base : **nombre** (entier ou réel, ex. 42, 3.14), **chaîne de caractères** (entre guillemets doubles ou simples, ex. "Yaoundé"), **booléen** (`true` ou `false`).
4. Point critique : `prompt()` renvoie toujours une chaîne (ou `null` si annulé). Avant tout calcul, convertis explicitement :
 - `parseInt(chaine, 10)` pour obtenir un **entier** (le second argument 10 force la base décimale, évitant les surprises octales);
 - `parseFloat(chaine)` pour obtenir un **nombre réel** (attention : il ne reconnaît que le point comme séparateur décimal, pas la virgule française).
5. Vérifie le type d'une variable avec l'opérateur `typeof` si un doute subsiste (ex. `typeof age` renvoie "number").

Réflexe Bac : dès qu'une valeur provient de `prompt()` et sert dans un calcul, écris immédiatement la conversion ; c'est le piège n°1 des sujets.

EXERCICE RÉSOLU 9 Variables et conversions

Énoncé. Une élève saisit son âge via `prompt("Quel est votre âge ?")` stocké dans la variable `a`.

1. Quel est le type de `a` après la saisie de la valeur 16 ?
2. On exécute `b = a + 2`; puis `c = parseInt(a, 10) + 2`; . Donner les valeurs de `b` et `c` en justifiant.
3. Écrire l'instruction qui calcule correctement l'âge de l'élève dans 2 ans.

SOLUTION

Solution détaillée.

1. La fonction `prompt()` renvoie toujours une **chaîne de caractères**. Donc `a` est de type chaîne et contient "16" (avec guillemets), même si l'utilisateur a tapé des chiffres.
2. Analysons les deux affectations :
 - `b = a + 2`; : l'opérateur `+` appliqué à une chaîne et un nombre déclenche la **concaténation** : le nombre 2 est transformé en chaîne et collé à "16". Donc `b = "162"` (chaîne).
 - `c = parseInt(a, 10) + 2`; : `parseInt("16", 10)` convertit la chaîne en l'entier 16; l'addition est alors arithmétique : `c = 16 + 2 = 18` (nombre).
3. L'instruction correcte est :

```
var ageDans2Ans = parseInt(a, 10) + 2;
```

Conclusion : sans conversion, `a + 2` donne la chaîne "162" et non 18 ; il faut donc toujours appliquer `parseInt()` (ou `parseFloat()` pour un réel) à une donnée issue de `prompt()` avant tout calcul.

ATTENTION Séparateur décimal : le piège de la virgule

En JavaScript, les littéraux numériques et les fonctions `parseFloat/parseInt` n'acceptent que le point comme séparateur décimal. Si un élève camerounais tape 12,5 (avec une virgule) dans un `prompt`, `parseFloat("12,5")` renverra 12 (il s'arrête à la virgule).

- Solution pédagogique : pour un convertisseur réel, remplacer la virgule par un point avant conversion : `parseFloat(saisie.replace(",", "."))`.
- Au Probatoire : les sujets utilisent généralement des valeurs entières ou des réels écrits avec un point dans l'énoncé. Sois vigilant(e) si tu dois tester ton propre code.

MÉTHODE Utiliser les opérateurs et évaluer une expression

1. **Arithmétiques** : + (addition/concaténation), -, *, /, et % (reste de la division euclidienne, appelé modulo).
2. **Comparaison** (résultat booléen) : == (égal, avec conversion de type), === (strictement égal, même type et même valeur — recommandé), != (différent), !== (strictement différent), <, >, <=, >=. Attention : = seul est une affectation, == est un test.
3. **Logiques** : && (ET : vrai si les deux conditions sont vraies), || (OU : vrai si au moins une condition est vraie), ! (NON).
4. **Affectations composées** : $x += 3$ équivaut à $x = x + 3$; de même $-=$, $*=$, $/=$. Les opérateurs ++ et -- ajoutent ou retranchent 1 : $x++$ équivaut à $x = x + 1$.
5. Pour évaluer une expression, respecte les priorités : parenthèses, puis *, /, %, puis +, -, puis comparaisons, puis &&, puis ||.

Réflexe Bac : pour une table d'exécution, déroule les instructions ligne par ligne en notant la valeur de chaque variable après chaque affectation.

EXERCICE RÉSOLU 10 Table d'exécution et opérateurs

Énoncé. On considère le script suivant :

```
var x = 10;
var y = 4;
x += y;
y++;
var r = x % y;
var test = (x > 12) && (y < 6);
```

Donner les valeurs finales de x, y, r et test, en détaillant chaque étape.

SOLUTION

Solution détaillée. On construit la table d'exécution ligne par ligne :

Instruction	Effet	Valeurs après exécution
var x = 10;	initialisation	x = 10
var y = 4;	initialisation	x = 10, y = 4
x += y;	équivaut à $x = x + y = 10 + 4$	x = 14, y = 4
y++;	équivaut à $y = y + 1$	x = 14, y = 5
var r = x % y;	reste de $14 \div 5$: $14 = 5 \times 2 + 4$	r = 4
var test = ...	(14 > 12) est vrai ; (5 < 6) est vrai ; vrai && vrai = vrai	test = true

Justifications : pour le modulo, la division euclidienne de 14 par 5 donne un quotient de 2 et un reste de 4, d'où $r = 4$. Pour le test logique, l'opérateur `&&` exige que les deux conditions soient vraies : c'est le cas ici. Conclusion : les valeurs finales sont $x = 14$, $y = 5$, $r = 4$ et `test = true`.

MÉTHODE Traduire un algorithme conditionnel ou itératif en JavaScript

1. Alternative : `if (condition) { ... } else { ... }`. La condition est entre **parenthèses**, les blocs entre **accolades**. Pour plusieurs cas, enchaîne `else if (condition) { ... }`.
2. Boucle bornée (nombre de tours connu) : `for (var i = 1; i <= n; i++) { ... }` avec les trois parties : initialisation ; condition de continuation ; incrémentation.
3. Boucle conditionnelle (on répète tant qu'une condition est vraie) : `while (condition) { ... }`. Vérifie qu'une instruction dans le corps fait évoluer la condition, sinon boucle infinie.
4. Traduction mot à mot depuis l'écriture algorithmique : « Si ... Alors ... Sinon ... FinSi » → `if/else`; « Pour i de 1 à n Faire ... FinPour » → `for`; « Tant que ... Faire ... FinTantQue » → `while`.
5. Teste ton script à la main sur 2 ou 3 valeurs avant de conclure.

EXERCICE RÉSOLU 11 Traduction d'un algorithme : mention au Probatoire

Énoncé. On donne l'algorithme suivant, qui affiche la mention d'un candidat selon sa moyenne m sur 20 :

Algorithme Mention
Variable m : reel
Debut
Ecrire("Entrer la moyenne :")
Lire(m)
Si $m \geq 10$ Alors
Ecrire("Admis")
Sinon
Ecrire("Refuse")
FinSi
Fin

1. Traduire cet algorithme en JavaScript (saisie par `prompt()`, affichage par `alert()`).
2. Que s'affiche-t-il si l'utilisateur saisit 12.5 (avec un point) ? Justifier.

SOLUTION

Solution détaillée.

1. Traduction ligne par ligne : « Ecrire + Lire » devient un `prompt()` (qui affiche le message et lit la réponse) ; la moyenne étant un réel, on convertit avec `parseFloat()` en gérant le séparateur décimal ; « Si/Sinon » devient `if/else` ; « Ecrire » devient `alert()`.

```
var saisie = prompt("Entrer la moyenne :");  
// Gestion du séparateur décimal (virgule ou point)  
var m = parseFloat(saisie.replace(",", "."));  
if (m >= 10){  
    alert("Admis");  
} else {  
    alert("Refuse");  
}
```

2. L'utilisateur saisit 12.5 (avec un point, notation JS standard). La fonction `prompt()` renvoie la chaîne "12.5", convertie par `parseFloat()` en le nombre 12,5. Le test $12,5 \geq 10$ est vrai : c'est donc le premier bloc qui s'exécute.

Conclusion : le script traduit fidèlement l'algorithme ; pour une moyenne de 12,5, la boîte de dialogue affiche « Admis ».

MÉTHODE Déclarer une fonction et l'appeler avec onclick

1. **Déclare** la fonction avec le mot-clé `function` (minuscules), suivi du nom, des paramètres entre parenthèses (éventuellement vides) et du corps entre accolades :
`function nom(param1, param2) { ... }`
2. Si la fonction doit fournir un résultat, utilise `return resultat;` ; sinon elle se contente d'exécuter des actions (affichages, modifications du DOM).
3. **Appelle** la fonction en écrivant son nom suivi des arguments entre parenthèses : `nom(5, 3);`. Une fonction non appelée ne s'exécute jamais.
4. Pour déclencher la fonction par un clic, ajoute l'événement `onclick` sur un bouton HTML : `<input type="button" value="Calculer" onclick="nom()">`.
5. Réflexe de rédaction : le nom de la fonction dans `onclick` doit être strictement identique (majuscules comprises) à celui de la déclaration.

EXERCICE RÉSOLU 12 Fonction et événement onclick

Énoncé. Un commerçant de Douala veut un bouton « Appliquer la remise » qui, lorsqu'on

clique dessus, calcule et affiche le prix d'un article de 50000 FCFA après une remise de 10 %.

1. Écrire la fonction JavaScript `remise()` qui effectue ce calcul et affiche le nouveau prix par `alert()`.
2. Écrire la balise HTML du bouton déclenchant cette fonction.

SOLUTION

Solution détaillée.

1. Calcul : une remise de 10 % sur 50000 FCFA vaut $50000 \times \frac{10}{100} = 5000$ FCFA ; le nouveau prix est donc $50000 - 5000 = 45000$ FCFA. On programme ce calcul de façon générale :

```
function remise() {  
    var prix = 50000;  
    var nouveauPrix = prix - prix * 10 / 100;  
    alert("Nouveau prix : " + nouveauPrix + " FCFA");  
}
```

Remarque : `prix * 10 / 100` est évalué de gauche à droite : $50000 \times 10 = 500000$, puis $500000 / 100 = 5000$; la soustraction donne bien 45000.

2. Le bouton utilise l'événement `onclick` pour appeler la fonction :

```
<input type="button" value="Appliquer la remise" onclick="remise()">
```

Conclusion : au clic sur le bouton, l'événement `onclick` déclenche l'appel de `remise()`, qui affiche « Nouveau prix : 45000 FCFA ».

MÉTHODE Programmer un convertisseur avec des zones de saisie

1. Crée le formulaire HTML : une zone de saisie `<input type="text" id="...">` pour la valeur à convertir, un bouton avec `onclick`, et éventuellement une zone pour le résultat (ex. `<output id="res">` ou `<input type="text" id="res" readonly>`).
2. Dans la fonction JavaScript, récupère la valeur saisie avec `document.getElementById("identifiant").value`. Cette valeur est une chaîne : convertis-la avec `parseFloat(saisie.replace(",", "."))` ou `parseInt(saisie, 10)`.
3. Applique la formule de conversion (par exemple $T_F = T_C \times 1,8 + 32$ pour les températures, ou montant $\times$ taux pour les devises).
4. Affiche le résultat : soit par `alert()`, soit en écrivant dans une zone de résultat avec `document.getElementById("resultat").value = ...` ou `.textContent = ...`.
5. Teste avec une valeur connue (ex. $0^\circ\text{C} = 32^\circ\text{F}$) pour valider la formule.

EXERCICE RÉSOLU 13 Convertisseur FCFA – Euro

Énoncé. Pour sa boutique en ligne, une commerçante de Bafoussam veut une page contenant une zone de saisie d'identifiant `montant` (somme en FCFA), un bouton « Convertir », et qui affiche par `alert()` l'équivalent en euros, sachant que 1 euro = 656 FCFA. Écrire le code HTML/JavaScript complet de la page.

SOLUTION

Solution détaillée.

1. Formule : si 1 € = 656 FCFA, alors un montant M en FCFA vaut $\frac{M}{656}$ euros.
2. Structure : zone de saisie `montant`, bouton dont l'`onclick` appelle `convertir()`, fonction qui lit, convertit, calcule et affiche.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Convertisseur FCFA – Euro</title>
    <script>
      function convertir() {
        // 1. Recuperation de la valeur saisie (chaîne)
        var saisie = document.getElementById("montant").value;
        // 2. Conversion en nombre reel (gestion virgule/point)
        var fcfa = parseFloat(saisie.replace(",", "."));
        // 3. Application de la formule
        var euro = fcfa / 656;
        // 4. Affichage du resultat (arrondi à 2 décimales pour lisibilité)
        alert(fcfa + " FCFA = " + euro.toFixed(2) + " euros");
      }
    </script>
  </head>
  <body>
    Montant en FCFA : <input type="text" id="montant">
    <input type="button" value="Convertir" onclick="convertir()">
  </body>
</html>
```

Vérification : pour $M = 6560$ FCFA saisis, `parseFloat("6560")` donne 6560, puis $6560/656 = 10$: la boîte affiche « 6560 FCFA = 10.00 euros », ce qui est exact. Conclusion : la page lit le montant saisi, le convertit grâce à `parseFloat()` (après remplacement virgule/point), applique la formule $\text{euro} = \text{fcfa}/656$ et affiche le résultat formaté au clic sur le bouton.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier la conversion après `prompt()` : `prompt()` renvoie une chaîne ; `a + b` concatène alors au lieu d'additionner ("`2`" + "`3`" donne "`23`", pas `5`). Toujours écrire `parseInt(saisie, 10)` ou `parseFloat(saisie.replace(",", "."))` avant un calcul.
2. Confondre `=` et `==` (ou `===`) : dans un `if`, on teste avec `===` (ex. `if (note === 10)`). Écrire `if (note = 10)` est une affectation : la variable vaut `10`, le test est vrai, la logique du programme est détruite.
3. Oublier les accolades ou les parenthèses : la condition d'un `if`, d'un `while` ou d'un `for` est toujours entre parenthèses, et le corps d'une fonction ou d'un bloc entre accolades. Une accolade manquante rend tout le script inutilisable (erreur de syntaxe).
4. Déclarer une fonction sans jamais l'appeler : une fonction ne s'exécute que si elle est appelée (par `onclick="maFonction()"` ou par une instruction d'appel `maFonction();`). Vérifier aussi que le nom dans `onclick` est identique, majuscules comprises, au nom déclaré (`function MaFonction` vs `onclick="mafonction()"` → erreur).
5. Se tromper de formule de conversion ou de sens du calcul : pour un convertisseur, poser d'abord la formule sur le brouillon (ex. `euro = FCFA / taux`, et non `FCFA × taux`) et la valider sur une valeur connue avant de rédiger le code.

9

EXERCICES

9.1 Je vérifie mes connaissances

EXERCICE 6 QCM : bases du langage

Pour chacune des affirmations suivantes, une seule proposition est correcte. Recopie le numéro et la lettre correspondante.

1. La balise HTML permettant d'insérer du code JavaScript dans une page est :
 - A. `<javascript>`
 - B. `<script>`
 - C. `<js>`
 - D. `<code>`
2. L'instruction `let x = "5" + 2;` affecte à `x` la valeur :
 - A. 7 (nombre)
 - B. "7" (chaîne)
 - C. "52" (chaîne)
 - D. Erreur de syntaxe
3. La fonction prédéfinie qui affiche une boîte de dialogue modale avec un message et un bouton «OK» est :
 - A. `prompt()`
 - B. `confirm()`
 - C. `alert()`
 - D. `console.log()`
4. Parmi les opérateurs suivants, lequel effectue une comparaison stricte (valeur et type)?
 - A. `==`
 - B. `===`
 - C. `=`
 - D. `!=`
5. L'instruction `parseInt("12.34px")` renvoie :
 - A. 12.34
 - B. 12

- C. NaN
- D. "12"

EXERCICE 7 Vrai ou Faux ? Justifie

Détermine si chacune des propositions est Vraie ou Fausse. Justifie brièvement ta réponse.

1. L'événement `onclick` se déclenche uniquement lorsque l'utilisateur double-clique sur un élément.
2. La fonction `prompt()` renvoie toujours une chaîne de caractères (`string`), même si l'utilisateur tape un nombre.
3. En JavaScript, les variables déclarées avec `var` ont une portée de bloc (limitées aux accolades `{ }`).
4. L'instruction `document.write("Bonjour")` écrit du contenu directement dans le flux HTML de la page ; si elle est exécutée après le chargement complet de la page, elle efface tout le contenu existant.
5. L'opérateur `&&` (ET logique) évalue son second opérande seulement si le premier est `true` (évaluation paresseuse).

EXERCICE 8 Vocabulaire et définitions

Complète les définitions suivantes en utilisant les termes appropriés : `Script`, Événement, Fonction, Variable, DOM, `parseFloat`.

1. Un _____ est un programme écrit en JavaScript, interprété par le navigateur pour rendre une page web interactive.
2. Une _____ est un conteneur nommé qui stocke une valeur (nombre, chaîne, booléen, objet, etc.) et dont le contenu peut changer pendant l'exécution.
3. Un _____ est une action de l'utilisateur (clic, frappe clavier, survol, chargement de page) ou une occurrence système à laquelle le script peut réagir.
4. Le _____ (Document Object Model) est la représentation arborescente de la page HTML que JavaScript peut parcourir et modifier.
5. La fonction _____ convertit une chaîne de caractères en nombre à virgule flottante (ex. "12.50" → 12.5).
6. Un bloc de code réutilisable, qui peut accepter des paramètres et retourner une valeur, s'appelle une _____.

EXERCICE 9 Prédiction de résultats

Sans exécuter le code, donne la valeur finale de la variable `resultat` pour chaque fragment indépendant. Explique le mécanisme de conversion de type (coercition) si nécessaire.

1. `let a = 10; let b = "5"; let resultat = a - b;`
2. `let x = "10"; let y = 5; let resultat = x + y;`
3. `let p = true; let q = false; let resultat = Number(p) + Number(q);`
4. `let s = "20"; let resultat = ++s; (Attention : pré-incrémentation sur une chaîne)`
5. `let n = parseFloat("3,14"); let resultat = n * 2; (Rappel : séparateur décimal JS = point)`

9.2 Je m'entraîne**EXERCICE 10 Table de multiplication interactive**

Écris une fonction JavaScript nommée `afficherTable()` qui :

1. Demande à l'utilisateur de saisir un entier `n` via `prompt()`.
2. Vérifie que la saisie est un nombre entier valide (utilise `Number.isInteger()` ou `parseInt()` + comparaison).
3. Si la saisie est invalide, affiche «Veillez entrer un nombre entier valide» via `alert()`.
4. Sinon, construit une chaîne de caractères contenant la table de multiplication de `n` de 1 à 10 (format : «`1 x 7 = 7`», une ligne par produit) en utilisant une boucle `for`.
5. Affiche le résultat final dans une seule boîte `alert()`.

Indication : utilise "

`\n`" pour les retours à la ligne dans la chaîne.

EXERCICE 11 Factorielle avec contrôle de saisie

Complète le script suivant pour qu'il calcule la factorielle d'un nombre saisi par l'utilisateur. Le script doit gérer les cas d'erreur (entrée non numérique, nombre négatif, nombre non entier) et afficher le résultat ou le message d'erreur avec `document.write()` dans le corps de la page.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
```

```
<title>Factorielle</title>
</head>
<body>
  <h1>Calcul de factorielle</h1>
  <script>
    // 1. Saisie utilisateur
    let saisie = prompt("Entrez un entier naturel n :");

    // 2. Conversion et validation (À COMPLÉTER)
    // let n = ...
    // if ( ... ) { document.write("Erreur : ..."); }
    // else { ... calcul factorielle ... }

  </script>
</body>
</html>
```

EXERCICE 12 Inversion de chaîne au clic

On dispose de la page HTML suivante. Écris le code JavaScript (à placer dans la balise `<script>`) pour que, au clic sur le bouton «Inverser», la chaîne saisie dans le champ `<input>` soit inversée et affichée dans le paragraphe `<p id="resultat">`.

- Utilise la méthode `split("")`, `reverse()`, `join("")`.
- Récupère l'élément `input` par son `id` et le paragraphe résultat par son `id`.
- Attache le gestionnaire d'événement via la propriété `onclick` de l'élément `button` (récupéré par `id`).

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Inverseur de texte</title>
</head>
<body>
  <h1>Inverseur de chaîne</h1>
  <label for="texte">Entrez un texte : </label>
  <input type="text" id="texte" placeholder="Ex : Yaoundé">
  <button id="btnInverser">Inverser</button>
  <p id="resultat"></p>

  <script>
    // TON CODE ICI
  </script>
</body>
```

```
</html>
```

EXERCICE 13 Convertisseur de températures : Kelvin → Celsius / Fahrenheit

1. Écris l'algorithme en pseudo-code (français structuré : Algorithme, Variable, Début, Si, Pour/TantQue, Fin) qui :

- Demande une température en Kelvin (K).
- Vérifie que $K \geq 0$ (zéro absolu). Si $K < 0$, affiche «Erreur : Température impossible».
- Sinon, calcule :

$$C = K - 273,15$$

$$F = C \times 1,8 + 32$$

- Affiche C et F avec 2 décimales.

2. Traduis cet algorithme en une fonction JavaScript `convertirKelvin()` déclenchée par un bouton `onclick`. Les entrées/sorties se font via `prompt()` et `alert()` (ou `document.write()`).

9.3 Je me prépare au Probatoire

EXERCICE 14 Type Bac : Débogage d'un convertisseur FCFA ↔ EUR

Un élève a écrit la page suivante pour convertir des Francs CFA en Euros et inversement. Le taux de change est fixé à 1 EUR = 655,957 FCFA. Le code comporte quatre erreurs majeures empêchant un fonctionnement correct.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Convertisseur FCFA / EUR</title>
</head>
<body>
  <h1>Conversion Monétaire (Banque des États de l'Afrique Centrale)</h1>
  <label>Montant : <input type="text" id="montant"></label><br><br>
  <label>
    <input type="radio" name="mode" value="fcfa2eur" checked> FCFA $to$ EUR
  </label>
```

```
<label>
  <input type="radio" name="mode" value="eur2fcfa"> EUR $\to$ FCFA
</label><br><br>
<button id="btnConvertir">Convertir</button>
<p id="affichage"></p>

<script>
  const TAUX = 655.957; // Erreur 1 : séparateur décimal ?

  // Erreur 2 : script bloquant, pas de fonction, pas d'écouteur d'événement
  let saisie = document.getElementById("montant").value;
  let mode = document.querySelector('input[name="mode"]:checked').value;

  if (mode == "fcfa2eur") { // Erreur 3 : opérateur de comparaison
    let res = saisie / TAUX;
    document.getElementById("affichage").innerHTML = res + " EUR";
  } else {
    let res = saisie * TAUX;
    document.getElementById("affichage").innerHTML = res + " FCFA";
  }
  // Erreur 4 : pas de conversion de type (parseFloat), pas de toFixed(2)
</script>
</body>
</html>
```

Questions :

1. Identifie et explique clairement les 4 erreurs numérotées dans les commentaires (et toute autre erreur que tu repères).
2. Propose le code JavaScript corrigé complet (à placer dans la balise `<script>`), structuré dans une fonction `convertir()` attachée à `onclick` du bouton. Le résultat doit être affiché avec 2 décimales (ex: `toFixed(2)`). Gère le cas où le champ est vide ou non numérique (`isNaN`).

EXERCICE 15 Programmation fonctionnelle : Somme des pairs avec `reduce`

On souhaite créer une fonction fléchée (arrow function) nommée `sommePairs` qui reçoit un tableau de nombres en paramètre et retourne la somme des seuls éléments pairs de ce tableau.

1. Écris la signature de la fonction fléchée.
2. Utilise la méthode `filter()` pour ne garder que les nombres pairs, puis `reduce()` pour calculer la somme. Rappel : un nombre n est pair si $n \% 2 === 0$.
3. Teste ta fonction avec le tableau `[12, 7, 25, 8, 30, 3, 10]` et affiche le résultat dans la console (`console.log()`). Quel est le résultat attendu?

4. Que retourne ta fonction si le tableau ne contient aucun nombre pair? (Teste avec `[1, 3, 5]`). Propose une modification pour qu'elle retourne `0` explicitement dans ce cas (valeur initiale de `reduce`).

EXERCICE 16 Mini-Quiz interactif : Culture Générale Cameroun

Programme un mini-quiz de 3 questions à choix multiples (QCM) qui s'exécute séquentiellement via `prompt()`. À la fin, affiche le score sur 3 avec `alert()`.

1. Données : Définit un tableau d'objets `questions`, chaque objet contenant : `texte` (string), `choix` (tableau de 3 strings), `bonneReponse` (index 0, 1 ou 2).
 - Q1 : «Capitale du Cameroun? » -> Choix : [«Douala », «Yaoundé », «Bafoussam »] -> Bonne : 1.
 - Q2 : «Monnaie officielle? » -> Choix : [«FCFA », «Euro », «Dollar »] -> Bonne : 0.
 - Q3 : «Mont le plus haut d'Afrique de l'Ouest? » -> Choix : [«Mont Cameroun », «Mont Kenya », «Kilimandjaro »] -> Bonne : 0.
2. Algorithme : Écrit le pseudo-code de la boucle qui parcourt le tableau, affiche la question et les choix numérotés (1, 2, 3) dans `prompt()`, lit la réponse (1, 2 ou 3), compare (attention : index JS base 0 vs affichage base 1), incrémente le score.
3. Code JS : Implémente la fonction `lancerQuiz()` et appelle-la. Gère le cas où l'utilisateur annule (`null`) ou tape une valeur invalide (ne compte pas comme bon, passe à la suivante).

10 CORRIGÉS DES EXERCICES

10.1 Corrigés — Je vérifie mes connaissances

CORRIGÉ Exercice 1 — QCM : bases du langage

1. Réponse B. La seule balise HTML valide pour insérer du JavaScript est `<script>...</script>`. Les balises `<javascript>`, `<js>` et `<code>` n'existent pas dans le standard HTML (ou n'ont pas ce rôle : `<code>` sert à afficher du code, pas à l'exécuter).
2. Réponse C. L'opérateur `+` avec une chaîne déclenche la **concaténation** : le nombre 2 est converti en chaîne "2", puis collé à "5" : résultat "52" (type `string`). C'est la coercion de type vers la chaîne.
3. Réponse C. `alert()` affiche une boîte modale avec un message et un seul bouton « OK ». `prompt()` demande une saisie, `confirm()` propose « OK » et « Annuler », `console.log()` écrit dans la console (invisible pour l'utilisateur).
4. Réponse B. `===` est l'égalité **stricte** : elle compare la valeur et le type sans conversion. `==` effectue une conversion préalable ("5" == 5 vaut `true`, mais "5" === 5 vaut `false`). `=` est l'affectation, `!=` la différence non stricte.
5. Réponse B. `parseInt()` lit les caractères de gauche à droite tant qu'ils forment un entier : il lit "12", s'arrête au point (non valide pour un entier) et renvoie le nombre 12. Le suffixe "px" est ignoré. `parseFloat("12.34px")` aurait renvoyé 12.34.

CORRIGÉ Exercice 2 — Vrai ou Faux ? Justifie

1. Faux. L'événement `onclick` se déclenche au simple clic. Le double-clic correspond à l'événement `ondblclick`.
2. Vrai. `prompt()` renvoie toujours une chaîne de caractères (`string`) quand l'utilisateur valide, ou `null` s'il annule. D'où la nécessité de `parseInt()` ou `parseFloat()` pour faire des calculs.
3. Faux. C'est l'inverse : `var` a une portée de fonction (ou globale), pas de bloc. Ce sont `let` et `const` qui ont une portée de bloc (limitée aux accolades `{ }`).
4. Vrai. `document.write()` écrit dans le flux HTML pendant le chargement. Appelée après la fin du chargement, elle rouvre le document et écrase tout le contenu existant de la page : c'est pourquoi on lui préfère `innerHTML`.
5. Vrai. `&&` est paresseux (short-circuit) : si le premier opérande est `false`, le résultat est

forcément `false` et le second opérande n'est pas évalué. De même, `||` n'évalue le second opérande que si le premier est `false`.

CORRIGÉ Exercice 3 — Vocabulaire et définitions

1. Un Script est un programme écrit en JavaScript, interprété par le navigateur.
2. Une Variable est un conteneur nommé qui stocke une valeur modifiable.
3. Un Événement est une action de l'utilisateur (clic, frappe, survol...) à laquelle le script peut réagir.
4. Le DOM (Document Object Model) est la représentation arborescente de la page HTML.
5. `parseFloat` convertit une chaîne en nombre à virgule flottante.
6. Un bloc de code réutilisable avec paramètres et valeur de retour s'appelle une Fonction.

CORRIGÉ Exercice 4 — Prédiction de résultats

1. `resultat = 5` (nombre). L'opérateur `-` n'a pas de sens sur les chaînes : JavaScript convertit `"5"` en nombre 5, puis calcule `10 - 5 = 5`.
2. `resultat = "105"` (chaîne). Avec `+`, la présence d'une chaîne impose la concaténation : 5 devient `"5"`, d'où `"10" + "5" = "105"`.
3. `resultat = 1.Number(true)` vaut 1 et `Number(false)` vaut 0 ; donc `1 + 0 = 1`.
4. `resultat = 21`. La pré-incrémentation `++s` convertit d'abord la chaîne `"20"` en nombre 20, l'incrémente à 21, puis affecte 21 à `resultat` (et à `s`).
5. `resultat = 6.parseFloat("3,14")` lit `"3"`, s'arrête devant la virgule (le séparateur décimal de JavaScript est le point) et renvoie 3. Donc `3 × 2 = 6`.

ATTENTION Point de vigilance

Le piège classique : `+` concatène, les autres opérateurs arithmétiques (`-`, `*`, `/`) convertissent en nombre. Et `parseFloat` n'accepte que le point décimal : `"3,14"` donne 3, il faut écrire `"3.14"` pour obtenir 3,14.

10.2 Corrigés — Je m'entraîne

CORRIGÉ Exercice 5 — Table de multiplication interactive

```
function afficherTable() {  
  // 1. Saisie (prompt renvoie toujours une chaîne ou null)  
  let saisie = prompt("Entrez un entier n :");  
  
  // 2. Conversion et validation  
  let n = Number(saisie);  
  if (saisie === null || saisie.trim() === "" || !Number.isInteger(n)) {  
    alert("Veuillez entrer un nombre entier valide");  
    return; // on quitte la fonction  
  }  
  
  // 3. Construction de la table (boucle for)  
  let message = "Table de " + n + " :\n";  
  for (let i = 1; i <= 10; i++) {  
    message += i + " x " + n + " = " + (i * n) + "\n";  
  }  
  
  // 4. Affichage en une seule boîte  
  alert(message);  
}
```

Trace pour $n = 7$: la boucle produit $1 \times 7 = 7$, $2 \times 7 = 14$, ..., $10 \times 7 = 70$, chaque ligne séparée par `\n`. La validation repose sur `Number.isInteger(n)` qui est `false` pour `NaN`, les décimaux et les conversions impossibles.

CORRIGÉ Exercice 6 — Factorielle avec contrôle de saisie

```
// 1. Saisie utilisateur  
let saisie = prompt("Entrez un entier naturel n :");  
  
// 2. Conversion et validation  
let n = Number(saisie);  
  
if (saisie === null || saisie.trim() === "" || isNaN(n)) {  
  document.write("Erreur : la saisie n'est pas numérique.");  
} else if (!Number.isInteger(n)) {  
  document.write("Erreur : n doit être un entier.");  
} else if (n < 0) {  
  document.write("Erreur : n doit être positif ou nul.");  
} else {  
  // 3. Calcul de la factorielle :  $n! = 1 \times 2 \times \dots \times n$   
  let fact = 1;
```

```

for (let i = 2; i <= n; i++){
    fact *= i; // equivalent a fact = fact * i
}
document.write(n + " ! = " + fact);
}

```

Vérification : $0! = 1$ (la boucle ne s'exécute pas, fact reste 1); $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$.
L'ordre des tests compte : on vérifie d'abord le caractère numérique, puis l'intégrité, puis le signe.

CORRIGÉ Exercice 7 — Inversion de chaîne au clic

```

// Recuperation des elements du DOM par leur id
let champTexte = document.getElementById("texte");
let bouton = document.getElementById("btnInverser");
let zoneResultat = document.getElementById("resultat");

// Attachement du gestionnaire d'evenement onclick
bouton.onclick = function() {
    let original = champTexte.value; // lecture de la saisie
    let inverse = original.split("") // chaîne -> tableau de caracteres
                                .reverse() // inversion du tableau
                                .join(""); // tableau -> chaîne
    zoneResultat.innerHTML = "Inverse : " + inverse;
};

```

Exemple : pour la saisie Yaoundé, `split("")` donne `["Y","a","o","u","n","d","é"]`, `reverse()` donne `["é","d","n","u","o","a","Y"]`, et `join("")` produit `édnuoaY`. Le script étant placé après les éléments HTML dans le `<body>`, le DOM est déjà construit au moment de l'exécution : `getElementById` fonctionne.

CORRIGÉ Exercice 8 — Convertisseur Kelvin → Celsius / Fahrenheit

1. Pseudo-code :

```

Algorithme ConvertisseurKelvin
Variable
    K, C, F : reel
Debut
    Ecrire("Entrez la temperature en Kelvin : ")
    Lire(K)
    Si K < 0 Alors
        Ecrire("Erreur : Temperature impossible")
    Sinon

```

```
C ← K - 273,15
F ← C * 1,8 + 32
Ecrire("Celsius : ", arrondi(C, 2))
Ecrire("Fahrenheit : ", arrondi(F, 2))
FinSi
Fin
```

2. Traduction JavaScript :

```
function convertirKelvin() {
  let saisie = prompt("Temperature en Kelvin :");
  let K = parseFloat(saisie);

  if (isNaN(K) || K < 0) {
    alert("Erreur : Temperature impossible");
    return;
  }
  let C = K - 273.15; // point decimal obligatoire en JS
  let F = C * 1.8 + 32;
  alert(K + " K = " + C.toFixed(2) + " C\n"
    + K + " K = " + F.toFixed(2) + " F");
}
```

Le bouton : `<button onclick="convertirKelvin()">Convertir</button>`.

Vérification numérique pour $K = 300$: $C = 300 - 273,15 = 26,85$; $F = 26,85 \times 1,8 + 32 = 48,33 + 32 = 80,33$. Pour $K = 273,15$: $C = 0$ et $F = 32$ (point de congélation de l'eau) : correct.

10.3 Corrigés — Je me prépare au Probatoire

CORRIGÉ Exercice 9 — Type Bac : Débogage du convertisseur FCFA ↔ EUR

1. Identification des erreurs :

1. Séparateur décimal : en JavaScript le séparateur est le point. `655.957` est en fait correct en JS ; l'erreur serait d'écrire `655,957` (virgule = syntaxe invalide ou interprétée comme opérateur virgule). Le commentaire invite à vérifier : la bonne écriture est bien `655.957`.
2. Script exécuté immédiatement : le code s'exécute au chargement, avant toute saisie : le champ est vide et l'utilisateur n'a jamais cliqué. Il faut encapsuler le traitement dans une fonction et l'attacher à l'événement `onclick` du bouton.
3. Comparaison : bien que `==` fonctionne ici (deux chaînes), la bonne pratique exigée est l'égalité stricte `===` (valeur et type, sans coercition).
4. Conversion de type absente : `.value` renvoie une chaîne. Sans `parseFloat()`, `saisie`

* TAUX fonctionne par coercion mais saisie vide ou invalide produit NaN; il faut aussi formater avec `toFixed(2)` et tester avec `isNaN()`.

Autre erreur repérable : le symbole \$ et les flèches → dans le HTML doivent être du texte pur, pas du LaTeX (dans la page réelle on écrit FCFA → EUR).

2. Code corrigé complet :

```
const TAUX = 655.957; // 1 EUR = 655,957 FCFA (point decimal)

function convertir() {
  // Lecture et conversion de la saisie
  let saisie = document.getElementById("montant").value;
  let montant = parseFloat(saisie);
  let affichage = document.getElementById("affichage");

  // Gestion des erreurs de saisie
  if (saisie.trim() === "" || isNaN(montant)) {
    affichage.innerHTML = "Erreur : entrez un montant numerique.";
    return;
  }

  // Lecture du mode choisi (bouton radio coché)
  let mode = document.querySelector('input[name="mode"]:checked').value;

  let res;
  if (mode === "fcfa2eur") { // comparaison stricte
    res = montant / TAUX;
    affichage.innerHTML = montant.toFixed(2) + " FCFA = "
      + res.toFixed(2) + " EUR";
  } else {
    res = montant * TAUX;
    affichage.innerHTML = montant.toFixed(2) + " EUR = "
      + res.toFixed(2) + " FCFA";
  }
}

// Attachement de l'événement onclick
document.getElementById("btnConvertir").onclick = convertir;
```

Vérification : 1000 FCFA → $1000 / 655,957 \approx 1,52$ EUR; 10 EUR → $10 \times 655,957 = 6559,57$ FCFA.

Barème indicatif (10 pts) : identification des 4 erreurs (4 pts); fonction + onclick (2 pts); `parseFloat` + `isNaN` (2 pts); `===` et `toFixed(2)` (1 pt); affichage correct via `innerHTML` (1 pt).

Points de vigilance : citer la portée temporelle du script (exécution au chargement vs au clic), justifier `parseFloat` par le type `string` de `.value`, et préciser que `toFixed` renvoie une chaîne formatée.

CORRIGÉ Exercice 10 — Somme des pairs avec `reduce`

1. et 2. Signature et implémentation :

```
const sommePairs = (tableau) => tableau
  .filter (n => n % 2 === 0)           // garde les pairs
  .reduce((acc, n) => acc + n, 0);    // somme, valeur initiale 0
```

3. Test :

```
console.log(sommePairs([12, 7, 25, 8, 30, 3, 10]));
```

`filter` conserve `[12, 8, 30, 10]`; `reduce` calcule $12 + 8 + 30 + 10 = 60$. Résultat attendu : 60.

4. Tableau sans pair : `sommePairs([1, 3, 5])` : `filter` renvoie `[]`. Grâce à la valeur initiale 0 passée en second argument de `reduce`, la fonction retourne 0 (sans cette valeur initiale, `reduce` sur un tableau vide lèverait une erreur `TypeError`). La modification demandée est donc précisément le , 0 déjà présent : il faut insister dessus dans la rédaction.

Point de vigilance : au Probatoire, cite explicitement le rôle de la valeur initiale de `reduce` et le test de parité `n % 2 === 0`.

CORRIGÉ Exercice 11 — Mini-Quiz interactif : Culture Générale Cameroun

1. Données :

```
const questions = [
  { texte: "Capitale du Cameroun ?",
    choix: ["Douala", "Yaounde", "Bafoussam"],
    bonneReponse: 1 },
  { texte: "Monnaie officielle ?",
    choix: ["FCFA", "Euro", "Dollar"],
    bonneReponse: 0 },
  { texte: "Mont le plus haut d'Afrique de l'Ouest ?",
    choix: ["Mont Cameroun", "Mont Kenya", "Kilimandjaro"],
    bonneReponse: 0 }
];
```

2. Pseudo-code :

```
Algorithme Quiz
Variable
  score : entier
  i, rep : entier
Debut
  score <- 0
  Pour i de 0 a taille(questions) - 1 Faire
    Afficher questions[i].texte et les choix numerotes 1, 2, 3
    Lire(rep)           // rep dans {1, 2, 3} ou annulation
    Si rep = questions[i].bonneReponse + 1 Alors
```

```
    score <- score + 1
  FinSi
FinPour
Ecrire("Score : ", score, "/3")
Fin
```

Le décalage +1 compense l'indexation : tableaux en base 0, choix affichés en base 1.

3. Implémentation JavaScript :

```
function lancerQuiz() {
  let score = 0;

  for (let i = 0; i < questions.length; i++) {
    // Construction du message : question + choix numerotes
    let message = questions[i].texte + "\n";
    for (let j = 0; j < questions[i].choix.length; j++) {
      message += (j + 1) + ". " + questions[i].choix[j] + "\n";
    }

    let saisie = prompt(message);

    // Annulation (null) ou valeur invalide : pas de point, on continue
    if (saisie === null) {
      continue;
    }
    let rep = parseInt(saisie);
    if (!isNaN(rep) && rep === questions[i].bonneReponse + 1) {
      score++;
    }
  }
  alert("Quiz termine ! Score : " + score + "/3");
}

lancerQuiz(); // appel de la fonction
```

Trace : réponses 2, 1, 1 (toutes correctes) → score = 3 ; si l'utilisateur annule la Q2 et se trompe à la Q3, score = 1. Les saisies non numériques (parseInt → NaN) sont neutralisées par le test isNaN.

Barème indicatif (10 pts) : tableau d'objets correct (2 pts) ; boucle de parcours (2 pts) ; gestion du décalage d'index +1 (2 pts) ; gestion de null et des saisies invalides (2 pts) ; affichage du score final (2 pts).

11 FICHE BILAN



Carte mentale : Les piliers de la programmation JavaScript en Première

FICHE BILAN L'essentiel à connaître pour l'examen

11.1 Définitions Clés

DÉFINITION Script JavaScript

Suite d'instructions écrites en langage JavaScript, interprétées par le navigateur côté client pour rendre une page Web interactive.

DÉFINITION Variable

Espace mémoire nommé (`let` ou `const`) stockant une valeur de type `Number`, `String`, `Boolean`, `Null`, `Undefined`, `Object`. Typage dynamique : le type change selon la valeur affectée.

DÉFINITION Événement

Action de l'utilisateur (clic, survol, frappe clavier, chargement page) détectée par le navigateur et pouvant déclencher l'exécution d'une fonction JavaScript.

DÉFINITION Fonction

Bloc de code réutilisable, identifié par un nom, acceptant des paramètres (entrées) et pouvant retourner une valeur (`return`). Déclenchée par un appel explicite ou un événement.

11.2 Syntaxe de Référence (À connaître par cœur)

Construction	Syntaxe / Exemple
Balise script	<code><script> ... </script></code> (dans <code><head></code> ou fin <code><body></code>) ou <code><script src="fichier.js"></script></code>
Variables	<code>let x = 10;</code> (modifiable) <code>const PI = 3.14;</code> (constante)
Affichage console	<code>console.log("Valeur :", x);</code>
Sorties boîte modale	<code>alert("Message");</code> <code>confirm("Valider ?");</code> (retourne true/false)
Saisie utilisateur	<code>let nom = prompt("Votre nom :");</code> (retourne String ou null)
Écriture dans page	<code>document.write("Texte " + variable);</code>
Conversion entier	<code>let n = parseInt("123");</code> (base 10 par défaut)
Conversion réel	<code>let x = parseFloat("3.14");</code>
Opérateurs arith.	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> Composées : <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>++</code> <code>--</code>
Opérateurs comp.	<code>===</code> <code>!==</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> (privilégier <code>===</code> pour type strict)
Opérateurs logiques	<code>&&</code> (ET) <code> </code> (OU) <code>!</code> (NON)
Condition if	<code>if (cond) {...} else if (cond) {...} else {...}</code>
Boucle for	<code>for (let i=0; i<10; i++) {...}</code>
Boucle while	<code>while (cond) {...} do {...} while (cond);</code>
Fonction déclarée	<code>function maFonction(param) { return val; }</code>
Gestion clic (HTML)	<code><button onclick="maFonction()">Cliquer</button></code>
Gestion clic (JS)	<code>btn.addEventListener('click', maFonction);</code>

11.3 Méthodes Express

MÉTHODE Écrire et tester un script simple

1. Créer un fichier `index.html` avec structure HTML5 valide.
2. Insérer `<script>` avant `</body>` (recommandé) ou dans `<head>`.
3. Déclarer variables avec `let/const`.
4. Utiliser `prompt()` pour entrées, `parseInt()/parseFloat()` pour convertir.
5. Traiter avec opérateurs et structures de contrôle.
6. Afficher résultat via `alert()`, `console.log()` ou `document.write()`.
7. Ouvrir dans navigateur, ouvrir outils développeur (F12) pour console/debug.

MÉTHODE Programmer un convertisseur (ex : Devises FCFA ↔ EUR)

1. Identifier données : taux fixe (ex : 1 EUR = 655,957 FCFA), sens de conversion.
2. Saisir montant via `prompt()` et convertir en `Number` (`parseFloat`).
3. Vérifier validité : `if (isNaN(montant) || montant <= 0)`.
4. Calculer : `resultat = montant * taux` ou `/ taux`.
5. Formater affichage : `resultat.toFixed(2)` pour 2 décimales.
6. Afficher via `alert()` ou injection dans le DOM (`element.textContent = ...`).

À RETENIR Checklist avant le Bac

- ☐ Je sais insérer du JavaScript dans une page HTML (`<script>` interne/externe).
- ☐ Je distingue `let` (variable) de `const` (constante) et je nomme correctement mes identificateurs (camelCase).
- ☐ Je maîtrise les 3 types primitifs principaux : `Number`, `String`, `Boolean` et le typage dynamique.
- ☐ Je convertis systématiquement les saisies `prompt()` avec `parseInt()` ou `parseFloat()`.
- ☐ J'utilise `===` (égalité stricte) au lieu de `==` pour éviter la coercition implicite.
- ☐ J'écris correctement les structures `if/else`, `for`, `while` avec accolades `{}` et indentation.
- ☐ Je sais déclarer une fonction avec paramètres et `return`, et l'appeler.
- ☐ Je lie une fonction à un clic via `onclick="maFonction()"` dans la balise HTML.
- ☐ Je conçois un convertisseur complet : saisie → conversion → test validité → calcul → affichage formaté.
- ☐ Je débogue avec `console.log()` et la console navigateur (F12).
- ☐ Je connais les limites : JS côté client seulement, pas d'accès direct BDD/serveur, désactivable par l'utilisateur.





Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+